CHAPTER 8

Generating Random Variates

Recommended sections for a first reading: 8.1 and 8.2

8.1 INTRODUCTION

A simulation that has any random aspects at all must involve sampling, or *generating*, random variates from probability distributions. As in Chap. 7, we use the phrase "generating a random variate" to refer to the activity of obtaining an observation on (or a realization of) a random variable from the desired distribution. These distributions are often specified as a result of fitting some appropriate distributional form, e.g., exponential, gamma, or Poisson, to observed data, as discussed in Chap. 6. In this chapter we assume that a distribution has already been specified somehow (including the values of the parameters), and we address the issue of how we can generate random variates with this distribution in order to run the simulation model. For example, the queueing-type models discussed in Sec. 1.4 and Chap. 2 required generation of interarrival and service times to drive the simulation through time, and the inventory model of Sec. 1.5 needed randomly generated demand sizes at the times when a demand occurred.

As we shall see in this chapter, the basic ingredient needed for *every* method of generating random variates from *any* distribution or random process is a source of IID U(0, 1) random variates. For this reason it is essential that a statistically reliable U(0, 1) random-number generator be available. Most computer installations and simulation packages have a convenient random-number generator but some of them (especially the older ones) do not perform adequately (see Chap. 7). Without an acceptable random-number generator, it is impossible to generate random variates correctly from any distribution. In the rest of this chapter, we therefore assume that a good source of random numbers is available.

There are usually several alternative algorithms that can be used for generating random variates from a given distribution, and several factors should be considered when choosing which algorithm to use in a particular simulation study. Unfortunately, these different factors often conflict with each other, so the analyst's judgment of which algorithm to use must involve a number of tradeoffs. All we can do here is raise some of the pertinent questions.

The first issue is *exactness*. We feel that, if possible, one should use an algorithm that results in random variates with exactly the desired distribution, within the unavoidable external limitations of machine accuracy and exactness of the U(0, 1) random-number generator. Efficient and exact algorithms are now available for all of the commonly used distributions, obviating the need to consider any older, approximate methods. [Many of these approximations, e.g., the well-known technique of obtaining a "normal" random variate as 6 less than the sum of 12 U(0, 1) random variates, are based on the central limit theorem.] On the other hand, the practitioner may argue that a specified distribution is really only an approximation to reality anyway, so that an approximate generation method should suffice; since this depends on the situation and is often difficult to quantify, we still prefer to use an exact method.

Given that we have a choice, then, of alternative exact algorithms, we would clearly like to use one that is *efficient*, in terms of both storage space and execution time. Some algorithms require *storage* of a large number of constants or of large tables, which could prove troublesome or at least inconvenient. As for execution time, there are really two factors. Obviously, we hope that we can accomplish the generation of each random variate in a small amount of time; this is called the *marginal execution time.* Second, some algorithms have to do some initial computing to specify constants or tables that depend on the particular distribution and parameters; the time required to do this is called the *setup time.* In most simulations, we shall be generating a large number of random variates from a given distribution, so that marginal execution time is likely to be more important than setup time. If the parameters of a distribution change often or randomly during the course of the simulation, however, setup time could become an important consideration.

A somewhat subjective issue in choosing an algorithm is its overall *complexity*, including conceptual as well as implementational factors. One must ask whether the potential gain in efficiency that might be experienced by using a more complicated algorithm is worth the extra effort to understand and implement it. This issue should be considered relative to the purpose in implementing a method for random-variate generation; a more efficient but more complex algorithm might be appropriate for use as permanent software but not for a "one-time" simulation model.

Finally, there are a few issues of a more technical nature. Some algorithms rely on a source of random variates from distributions other than U(0, 1), which is undesirable, other things being equal. Another technical issue is that a given algorithm may be efficient for some parameter values but costly for others. We would like to have algorithms that are efficient for all parameter values (sometimes called *robustness* of the algorithm); see Devroye (1988). One last technical point is relevant if we want to use certain kinds of variance-reduction techniques in order to obtain better (less variable) estimates (see Chap. 11 and also Chaps. 10 and 12). Two commonly

used variance-reduction techniques (common random numbers and antithetic variates) require synchronization of the basic U(0, 1) input random variates used in the simulation of the system(s) under study, and this synchronization is more easily accomplished for certain types of random-variate generation algorithms. In particular, the general inverse-transform approach can be very helpful in facilitating the desired synchronization and variance reduction; Sec. 8.2.1 treats this point more precisely.

There are a number of comprehensive references on random-variate generation, including Dagpunar (1988), Devroye (1986), Fishman (1996), Gentle (2010), Hörmann et al. (2004), and Johnson (1987). There are also several computer packages that provide good capabilities for generating random variates from a wide variety of distributions, such as the IMSL routines [Rogue Wave (2013)] and the C codes in secs. 7.1 and 7.3 of Press et al. (2007).

The remainder of this chapter is organized as follows. In Sec. 8.2 we survey the most important general approaches for random-variate generation, including examples and general discussions of the relative merits of the various approaches. In Secs. 8.3 and 8.4 we present algorithms for generating random variates from particular continuous and discrete distributions that have been found useful in simulation. Finally, in Secs. 8.5 and 8.6 we discuss two more specialized topics: generating correlated random variates and generating realizations of both stationary and non-stationary arrival processes.

8.2 GENERAL APPROACHES TO GENERATING RANDOM VARIATES

There are many techniques for generating random variates, and the particular algorithm used must, of course, depend on the distribution from which we wish to generate; however, nearly all these techniques can be classified according to their theoretical basis. In this section we discuss these general approaches.

8.2.1 Inverse Transform

Suppose that we wish to generate a random variate X that is continuous (see Sec. 4.2) and has distribution function F that is continuous and strictly increasing when $0 < F(x) < 1$. [This means that if $x_1 < x_2$ and $0 < F(x_1) \leq F(x_2) < 1$, then in fact $F(x_1) < F(x_2)$.] Let F^{-1} denote the inverse of the function F. Then an algorithm for generating a random variate X having distribution function F is as follows (recall that $\sim$ is read "is distributed as"):

1. Generate $U \sim \text{U}(0, 1)$.
2. Return $X = F^{-1}(U)$.

Note that $F^{-1}(U)$ will always be defined, since $0 \leq U \leq 1$ and the range of F is [0, 1]. Figure 8.1 illustrates the algorithm graphically, where the random variable

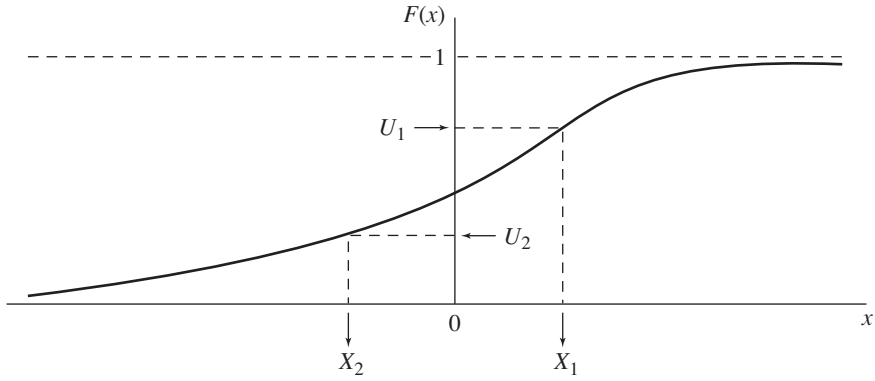


FIGURE 8.1
Inverse-transform method for continuous random variables.

corresponding to this distribution function can take on either positive or negative values; the particular value of U determines which will be the case. In the figure, the random number U_1 results in the positive random variate X_1, while the random number U_2 leads to the negative variate X_2.

To show that the value X returned by the above algorithm, called the general *inverse-transform method*, has the desired distribution F, we must show that for any real number x, $P(X \leq x) = F(x)$. Since F is invertible, we have

$$P(X \leq x) = P(F^{-1}(U) \leq x) = P(U \leq F(x)) = F(x)$$

where the last equality follows since $U \sim \text{U}(0, 1)$ and $0 \leq F(x) \leq 1$. (See the discussion of the uniform distribution in Sec. 6.2.2.)

EXAMPLE 8.1. Let X have the exponential distribution with mean β (see Sec. 6.2.2). The distribution function is

$$F(x) = \begin{cases} 1 - e^{-x/\beta} & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

so to find F^{-1}, we set $u = F(x)$ and solve for x to obtain

$$F^{-1}(u) = -\beta \ln (1 - u)$$

Thus, to generate the desired random variate, we first generate a $U \sim \text{U}(0, 1)$ and then let $X = -\beta \ln U$. [It is possible in this case to use U instead of $1 - U$, since $1 - U$ and U have the same U(0, 1) distribution. This saves a subtraction.]

In the above example, we replaced $1 - U$ by U for the sake of a perhaps minor gain in efficiency. However, replacing $1 - U$ by U in situations like this results in negative correlation of the X's with the U's, rather than positive correlation. Also, it is not true that wherever a "$1 - U$" appears in a variate-generation algorithm it can be replaced by a "U," as illustrated in Sec. 8.3.15.

The inverse-transform method's validity in the continuous case was demonstrated mathematically above, but there is also a strong intuitive appeal. The density function $f(x)$ of a continuous random variable may be interpreted as the relative chance of observing variates on different parts of the range; on regions of the x axis above which $f(x)$ is high we expect to observe a lot of variates, and where $f(x)$ is low we should find only a few. For example, Fig. 8.2*b* shows the density function for the Weibull distribution with shape parameter $\alpha = 1.5$ and scale parameter $\beta = 6$ (see Sec. 6.2.2 for definition of this distribution), and we would expect that many generated variates would fall between, say, $x = 2$ and $x = 5$, but not many between 13 and 16. Figure 8.2*a* shows the corresponding distribution function, $F(x)$. Since the density is the derivative of the distribution function [that is, $f(x) = F'(x)$], we can view $f(x)$ as the "slope function" of $F(x)$; that is, $f(x)$ is the slope of F at x. Thus, F rises most steeply for values of x where $f(x)$ is large (e.g., for x between 2 and 5), and, conversely, F is relatively flat in regions where $f(x)$ is small (e.g., for x between 13 and 16). Now, the inverse-transform method says to take the random number U, which should be evenly (uniformly) spread on the interval [0, 1] on the vertical axis of the plot for $F(x)$, and "read across and down." More U's will hit the steep parts of $F(x)$ than the flat parts, thus concentrating the X's under those regions where $F(x)$ is steep—which are precisely those regions where $f(x)$ is high. The interval [0.25, 0.30] on the vertical axis in Fig. 8.2*a* should contain about 5 percent of the U's, which lead to X's in the relatively narrow region [2.6, 3.0] on the x axis; thus, about 5 percent of the X's will be in this region. On the other hand, the interval [0.93, 0.98] on the vertical axis, which is the same size as [0.25, 0.30] and thus contains about 5 percent of the U's as well, leads to X's in the large interval [11.5, 14.9] on the x axis; here we will also find about 5 percent of the X's, but spread out sparsely over a much larger interval.

Figure 8.3 shows the algorithm in action, with 50 X's being generated (using the random-number generator from App. 7A with stream 1). The U's are plotted on the vertical axis of Fig. 8.3*a*, and the X's corresponding to them are obtained by following the dashed lines across and down. Note that the U's on the vertical axis are fairly evenly spread, but the X's on the horizontal axis are indeed more dense where the density function $f(x)$ is high, and become more spread out where $f(x)$ is low. Thus, the inverse-transform method essentially deforms the uniform distribution of the U's to result in a distribution of the X's in accordance with the desired density.

The inverse-transform method can also be used when X is discrete. Here the distribution function is

$$F(x) = P(X \le x) = \sum_{x_i \le x} p(x_i)$$

where $p(x_i)$ is the probability mass function

$$p(x_i) = P(X = x_i)$$

(We assume that X can take on only the values $x_1, x_2, \ldots$ where $x_1 < x_2 < \cdots$.) Then the algorithm is as follows:

1. Generate $U \sim \mathrm{U}(0, 1)$.
2. Determine the smallest positive integer I such that $U \le F(x_I)$, and return $X = x_I$.

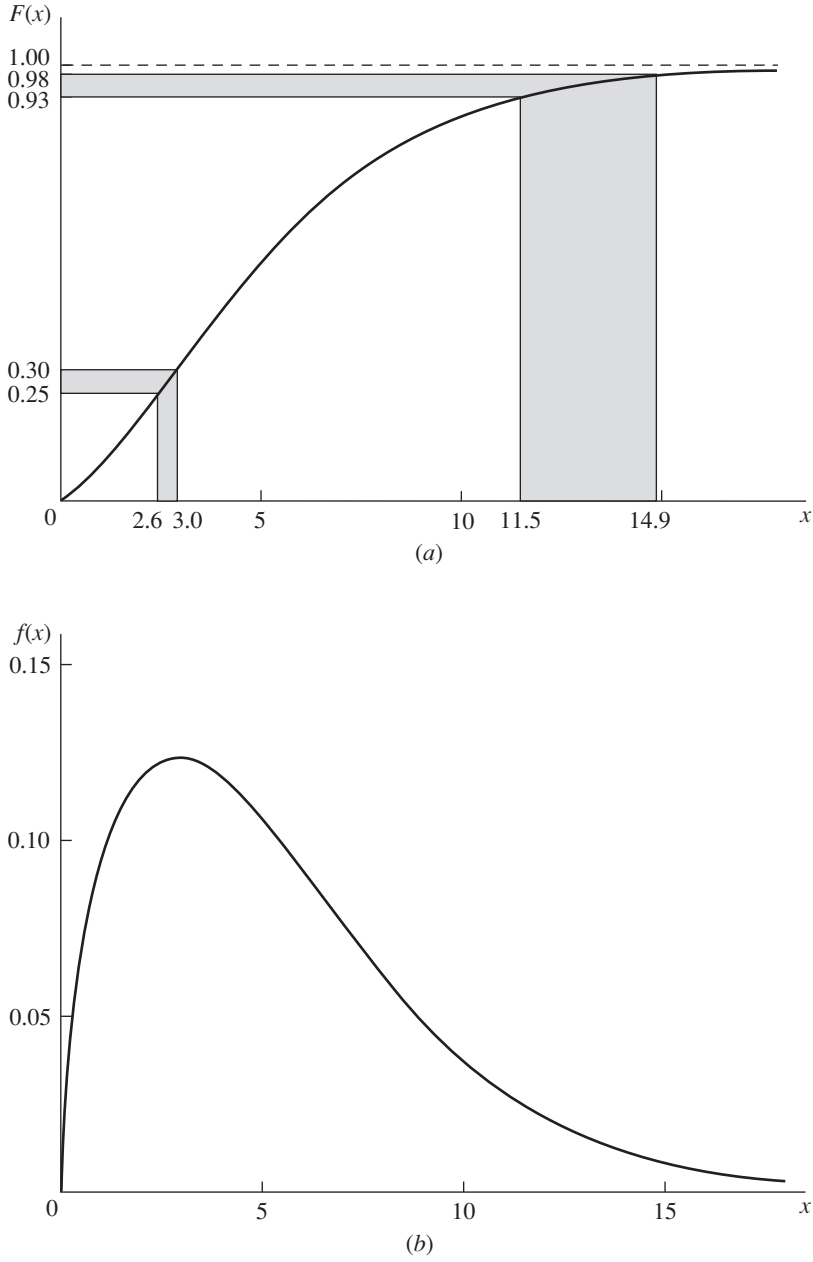
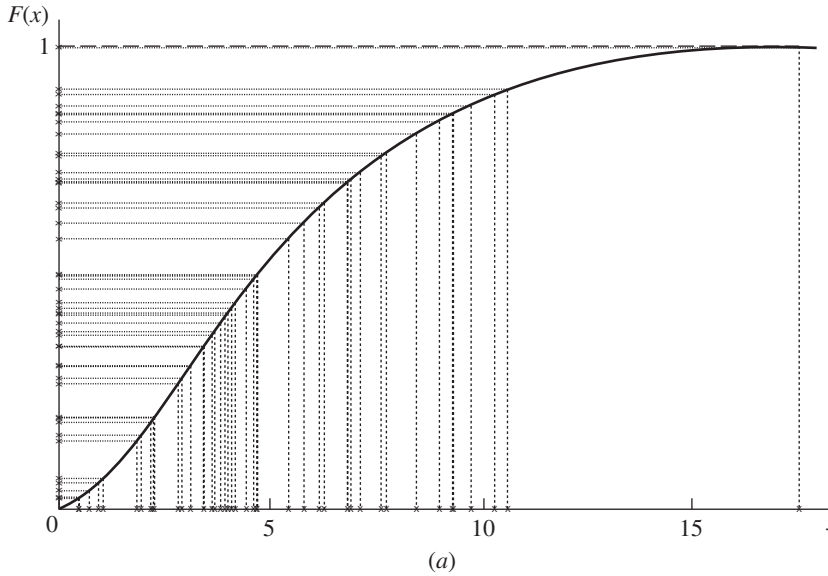


FIGURE 8.2
(*a*) Intervals for U and X, inverse transform for Weibull(1.5, 6) distribution; (*b*) density for Weibull(1.5, 6) distribution.



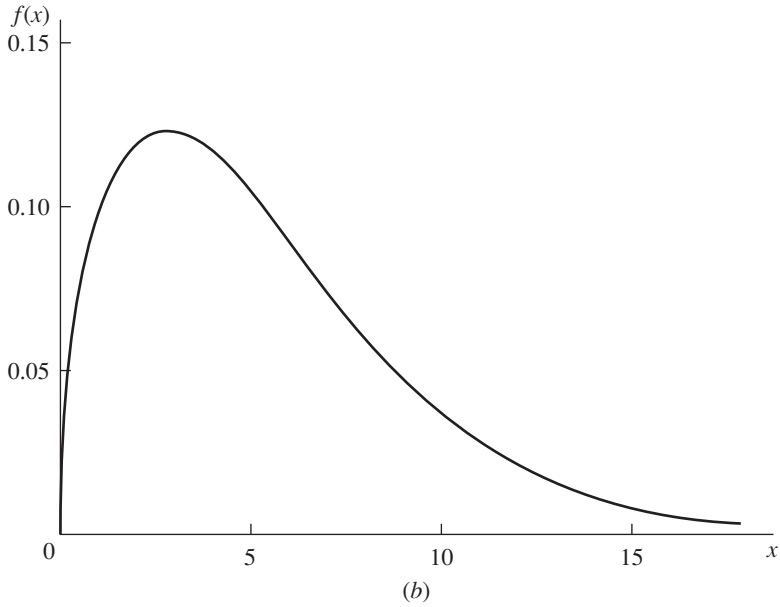


FIGURE 8.3
(*a*) Sample of 50 *U*'s and *X*'s, inverse transform for Weibull(1.5, 6) distribution; (*b*) density for Weibull(1.5, 6) distribution.

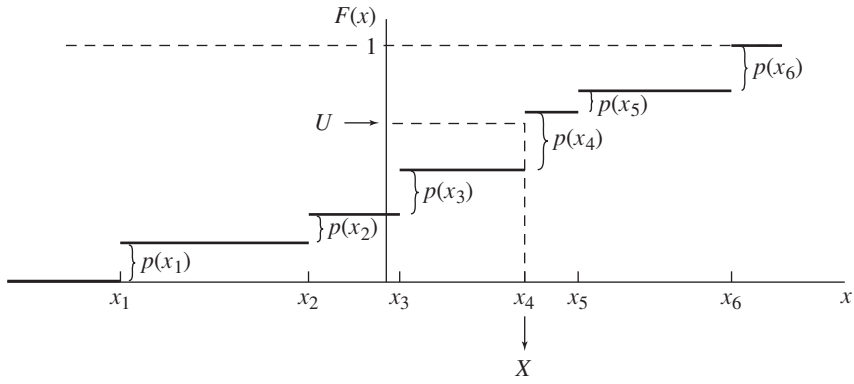


FIGURE 8.4
Inverse-transform method for discrete random variables.

Figure 8.4 illustrates the method, where we generate $X = x_4$ in this case. Although this algorithm might not seem related to the inverse-transform method for continuous random variates, the similarity between Figs. 8.1 and 8.4 is apparent.

To verify that the discrete inverse-transform method is valid, we need to show that $P(X = x_i) = p(x_i)$ for all i. For $i = 1$, we get $X = x_1$ if and only if $U \le F(x_1) = p(x_1)$, since we have arranged the x_i's in increasing order. Since $U \sim \text{U}(0, 1)$, $P(X = x_1) = p(x_1)$, as desired. For $i \ge 2$, the algorithm sets $X = x_i$ if and only if $F(x_{i-1}) < U \le F(x_i)$, since the i chosen by the algorithm is the smallest positive integer such that $U \le F(x_i)$. Further, since $U \sim \text{U}(0, 1)$ and $0 \le F(x_{i-1}) < F(x_i) \le 1$,

$$P(X = x_i) = P[F(x_{i-1}) < U \le F(x_i)] = F(x_i) - F(x_{i-1}) = p(x_i)$$

EXAMPLE 8.2. Recall the inventory example of Sec. 1.5, where the demand-size random variable X is discrete, taking on the values 1, 2, 3, 4 with respective probabilities $\frac{1}{6}$, $\frac{1}{3}$, $\frac{1}{3}$, $\frac{1}{6}$; the distribution function F is given in Fig. 4.2. To generate an X, first generate $U \sim \text{U}(0, 1)$ and set X to either 1, 2, 3, or 4, depending on the subinterval in [0, 1] into which U falls. If $U \le \frac{1}{6}$, then let $X = 1$; if $\frac{1}{6} < U \le \frac{1}{2}$, let $X = 2$; if $\frac{1}{2} < U \le \frac{5}{6}$, let $X = 3$; finally, if $\frac{5}{6} < U$, let $X = 4$.

Although both Fig. 8.4 and Example 8.2 deal with discrete random variables taking on only finitely many values, the discrete inverse-transform method can also be used directly as stated to generate random variates with an infinite range, e.g., the Poisson, geometric, or negative binomial.

The discrete inverse-transform method, when written as in Example 8.2, is really quite intuitive. We split the unit interval into contiguous subintervals of width $p(x_1)$, $p(x_2)$, . . . and assign X according to whichever of these subintervals contains the generated U. For example, U will fall in the second subinterval with probability $p(x_2)$, in which case we let $X = x_2$. The efficiency of the algorithm will depend on how we look for the subinterval that contains a given U. The simplest approach would be to start at the left and move up; first check whether $U \le p(x_1)$, in which case we return $X = x_1$. If $U > p(x_1)$, check whether $U \le p(x_1) + p(x_2)$, in which case we return $X = x_2$, etc. The number of comparisons needed to determine a value for X is

thus dependent on U and the $p(x_i)$'s. If, for example, the first several $p(x_i)$'s are very small, the probability is high that we will have to do a large number of comparisons before the algorithm terminates. This suggests that we might be well advised to perform this search in a more sophisticated manner, using appropriate sorting and searching techniques from the computer-science literature [see, for example, Knuth (1998b)]. One simple improvement would be first to check whether U lies in the widest subinterval, since this would be the single most-likely case. If not, we would check the second widest subinterval, etc. This method would be particularly useful when some $p(x_i)$ values are considerably greater than others and there are many x_i's; see Prob. 8.2 for more on this idea. See also Chen and Asau (1974) and Fishman and Moore (1984) for very efficient search methods using the idea of *indexing*.

Generalization, Advantages, and Disadvantages of the Inverse-Transform Method

Both the continuous and discrete versions of the inverse-transform method can be combined, at least formally, into the more general form

$$X = \min\{x: F(x) \geq U\}$$

which has the added advantage of being valid for distributions that are *mixed*, i.e., have both continuous and discrete components, as well as for continuous distribution functions with flat spots. To check that the above is valid in the continuous case, note in Fig. 8.1 that the set $\{x: F(x) \geq U_1\}$ is the interval $[X_1, \infty)$, which has minimum X_1. In the discrete case, we see in Fig. 8.4 that $\{x: F(x) \geq U\} = [x_4, \infty)$, which has minimum x_4. Figure 8.5 shows a mixed distribution with two jump

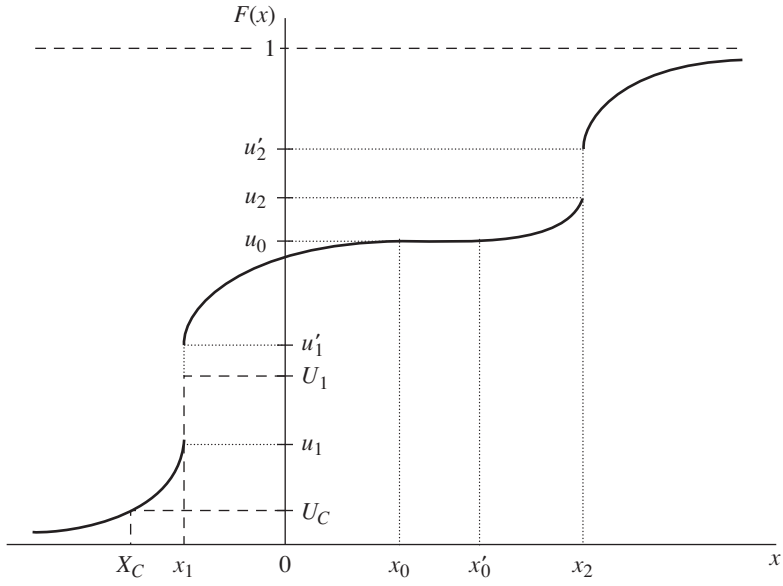


FIGURE 8.5
Inverse-transform method for a mixed distribution.

discontinuities and a flat spot; in this case the associated random variable X should satisfy $P(X = x_1) = u_1' - u_1$ (jump at x_1), $P(X = x_2) = u_2' - u_2$ (jump at x_2), and $P(x_0 \le X \le x_0') = 0$ (flat spot between x_0 and x_0'). For the continuous component, note that

$$X = \min\{x: F(x) \ge U_C\} = \min[X_C, \infty) = X_C$$

as expected. For the jump discontinuity at x_1, we get, for $u_1 \le U_1 \le u_1'$,

$$X = \min\{x: F(x) \ge U_1\} = \min[x_1, \infty) = x_1$$

which will occur with probability $u_1' - u_1$, as desired; the jump at x_2 is similar. For the flat spot, we will generate a variate X in (x_0, x_0') only if we generate a random number U that is *equal* to u_0; as U represents a continuous random variable, this occurs with probability 0, although in practice the finite accuracy of the generated random number U could result in $U = u_0$. Thus, this more general statement of the inverse-transform method handles any continuous, discrete, or mixed distribution. How it is actually implemented, though, will of course depend heavily on the distribution desired.

Let us now consider some general advantages and disadvantages of the inverse-transform method in both the continuous and discrete cases. One possible impediment to use of this method in the continuous case is the need to evaluate $F^{-1}(U)$. Since we might not be able to write a formula for F^{-1} in closed form for the desired distribution (e.g., the normal and gamma distributions), simple use of the method, as in Example 8.1, might not be possible. However, even if F^{-1} does not have a simple closed-form expression, we might be able to use numerical methods, e.g., a power-series expansion, to evaluate F^{-1}. (See, e.g., the discussion in Sec. 8.3 concerning the generation of gamma, normal, and beta random variates.) These numerical methods can yield arbitrary accuracy, so in particular can match the accuracy inherent in machine roundoff error; in this sense, they are exact for all practical purposes. However, Devroye (1986, pp. 31–35) points out that it may be difficult to specify an acceptable stopping rule for some distributions, especially those whose range is infinite. Kennedy and Gentle (1980, chap. 5) provide a comprehensive survey of numerical methods for computing distribution functions and their inverses; see also Abramowitz and Stegun (1964, chap. 26) and Press et al. (2007, chap. 6). The IMSL library [Rogue Wave (2013)] includes routines to compute most of the common distribution functions and their inverses, using carefully chosen algorithms. As an alternative to approximating F^{-1} numerically, Marsaglia (1984) proposes that a function g be found that is "close" to F^{-1} and is easy to evaluate; then X is generated as $g(Y)$, where Y has a particular distribution that is "close" to U(0, 1). Marsaglia called this method the *exact-approximation method* [see also Fishman (1996, pp. 185–187)].

Hörmann and Leydold (2003) proposed a general adaptive method for constructing a highly accurate Hermite interpolation for F^{-1} in the case of continuous distributions. Based on a one-time setup, their method produces moderate-size tables for the interpolation points and coefficients. Generating random variates using these tables is then very fast. The code for their method is available in a public-domain library called UNURAN.

A second potential disadvantage is that for a given distribution the inverse-transform method may not be the fastest way to generate the corresponding random variate; in Secs. 8.3 and 8.4 we discuss the efficiency of alternative algorithms for each distribution considered.

Despite these possible drawbacks, there are some important advantages in using the inverse-transform method. The first is to facilitate variance-reduction techniques (see Chap. 11) that rely on inducing correlation between random variates; examples of such techniques are common random numbers and antithetic variates. If F_1 and F_2 are two distribution functions, then $X_1 = F_1^{-1}(U_1)$ and $X_2 = F_2^{-1}(U_2)$ will be random variates with respective distribution functions F_1 and F_2, where U_1 and U_2 are random numbers. If U_1 and U_2 are independent, then of course X_1 and X_2 will be independent as well. However, if we let $U_2 = U_1$, then the correlation between X_1 and X_2 is made as positive as possible, and taking $U_2 = 1 - U_1$ (which, recall, is also distributed uniformly over [0, 1]) makes the correlation between X_1 and X_2 as negative as possible. Thus, the inverse-transform method induces the strongest correlation (of either sign) between the generated random variates, which we hope will propagate through the simulation model to induce the strongest possible correlation in the output, thereby contributing to the success of the variance-reduction technique. [It is possible, however, to induce correlation in random variates generated by methods other than the inverse-transform method; see Schmeiser and Kachitvichyanukul (1990)]. On a more pragmatic level, inverse transform eases application of variance-reduction techniques since we always need exactly one random number to produce one value of the desired X. (Other methods to be discussed later may require several random numbers to obtain a single value of X, or the number of random numbers might itself be random, as in the acceptance-rejection method.) This observation is important since proper implementation for many variance-reduction techniques requires some sort of synchronization of the input random numbers between different simulation runs. If the inverse-transform technique is used, synchronization is easier to achieve.

The second advantage concerns ease of generating from truncated distributions (see Sec. 6.8). In the continuous case, suppose that we have a density f with corresponding distribution function F. For $a < b$ (with the possibility that $a = -\infty$ or $b = +\infty$), we define the *truncated density*

$$f^*(x) = \begin{cases} \dfrac{f(x)}{F(b) - F(a)} & \text{if } a \le x \le b \\ 0 & \text{otherwise} \end{cases}$$

which has corresponding *truncated distribution function*

$$F^*(x) = \begin{cases} 0 & \text{if } x < a \\ \dfrac{F(x) - F(a)}{F(b) - F(a)} & \text{if } a \le x \le b \\ 1 & \text{if } b < x \end{cases}$$

(The discrete case is analogous.) Then an algorithm for generating an X having distribution function F^* is as follows:

1. Generate $U \sim \mathrm{U}(0, 1)$.
2. Let $V = F(a) + [F(b) - F(a)]U$.
3. Return $X = F^{-1}(V)$.

We leave it as an exercise (Prob. 8.3) to show that the X defined by this algorithm indeed has distribution function F^*. Note that the inverse-transform idea is really used twice: first in step 2 to distribute V uniformly between $F(a)$ and $F(b)$ and then in step 3 to obtain X. (See Prob. 8.3 for another way to generate X and Prob. 8.4 for a different type of truncation, which results in a distribution function that is *not* the same as F^*.)

Finally, the inverse-transform method can be quite useful for generating order statistics. Suppose that $Y_1, Y_2, \ldots, Y_n$ are IID with common distribution function F and that for $i = 1, 2, \ldots, n$, $Y_{(i)}$ denotes the ith smallest of the Y_j's. Recall from Chap. 6 that $Y_{(i)}$ is called the ith order statistic from a sample of size n. [Order statistics have been useful in simulation when one is concerned with the reliability, or *lifetime*, of some system having components subject to failure. If Y_j is the lifetime of the jth component, then $Y_{(1)}$ is the lifetime of a system consisting of n such components connected in series and $Y_{(n)}$ is the lifetime of the system if the components are connected in parallel.] One direct way of generating $X = Y_{(i)}$ is first to generate n IID variates $Y_1, Y_2, \ldots, Y_n$ with distribution function F, then sort them into increasing order, and finally set X to the ith value of the Y_j's after sorting. This method, however, requires generating n separate variates with distribution function F and then sorting them, which can be slow if n is large. As an alternative, we can use the following algorithm to generate $X = Y_{(i)}$:

1. Generate $V \sim \mathrm{beta}(i, n - i + 1)$.
2. Return $X = F^{-1}(V)$.

The validity of this algorithm is established in Prob. 8.5. Note that step 1 requires generating from a beta distribution, which we discuss below in Sec. 8.3.8. No sorting is required, and we need to evaluate F^{-1} only once; this is particularly advantageous if n is large or evaluating F^{-1} is slow. Two important special cases are generating either the minimum or maximum of the n Y_j's, where step 1 becomes particularly simple. For the minimum, $i = 1$ and V in step 1 can be defined by $V = 1 - U^{1/n}$, where $U \sim \mathrm{U}(0, 1)$. For the maximum, $i = n$ and we can set $V = U^{1/n}$ in step 1. (See Prob. 8.5 for verification in these two special cases.) For more on generating order statistics, see Ramberg and Tadikamalla (1978), Schmeiser (1978a, 1978b), and Schucany (1972).

8.2.2 Composition

The *composition* technique applies when the distribution function F from which we wish to generate can be expressed as a convex combination of other distribution functions $F_1, F_2, \ldots$. We would hope to be able to sample from the F_j's more easily than from the original F.

Specifically, we assume that for all x, $F(x)$ can be written as

$$F(x) = \sum_{j=1}^{\infty} p_j F_j(x)$$

where $p_j \geq 0$, $\sum_{j=1}^{\infty} p_j = 1$, and each F_j is a distribution function. (Although we have written this combination as an infinite sum, there may be a k such that $p_k > 0$ but $p_j = 0$ for $j > k$, in which case the sum is actually finite.) Equivalently, if X has density f that can be written as

$$f(x) = \sum_{j=1}^{\infty} p_j f_j(x)$$

where the f_j's are other densities, the method of composition still applies; the discrete case is analogous. The general composition algorithm, then, is as follows:

1. Generate a positive random integer J such that

$$P(J = j) = p_j \qquad \text{for } j = 1, 2, \ldots$$

2. Return X with distribution function F_J.

Step 1 can be thought of as choosing the distribution function F_j with probability p_j and could be accomplished, for example, by the discrete inverse-transform method. Given that $J = j$, generating X in step 2 should be done, of course, independently of J. By conditioning on the value of J generated in step 1, we can easily see that the X returned by the algorithm will have distribution function F [see, for example, Ross (2003, chap. 3)]:

$$P(X \leq x) = \sum_{j=1}^{\infty} P(X \leq x \mid J = j) P(J = j) = \sum_{j=1}^{\infty} F_j(x) p_j = F(x)$$

Sometimes we can give a geometric interpretation to the composition method. For a continuous random variable X with density f, for example, we might be able to divide the area under f into regions of areas $p_1, p_2, \ldots$, corresponding to the decomposition of f into its convex-combination representation. Then we can think of step 1 as choosing a region and step 2 as generating from the distribution corresponding to the chosen region. The following two examples allow this kind of geometric interpretation.

EXAMPLE 8.3. The *double-exponential* (or *Laplace*) *distribution* has density $f(x) = 0.5e^{-|x|}$ for all real x; this density is plotted in Fig. 8.6. From the plot we see that except for the normalizing factor 0.5, $f(x)$ is two exponential densities placed back to back; this suggests the use of composition. Indeed, we can express the density as

$$f(x) = 0.5e^{x} I_{(-\infty,0)}(x) + 0.5e^{-x} I_{[0,\infty)}(x)$$

where I_A denotes the *indicator function* of the set A, defined by

$$I_A(x) = \begin{cases} 1 & \text{if } x \in A \\ 0 & \text{otherwise} \end{cases}$$

Thus, $f(x)$ is a convex combination of $f_1(x) = e^{x} I_{(-\infty,0)}(x)$ and $f_2(x) = e^{-x} I_{[0,\infty)}(x)$, both of which are densities, and $p_1 = p_2 = 0.5$. Therefore, we can generate an X with density f by

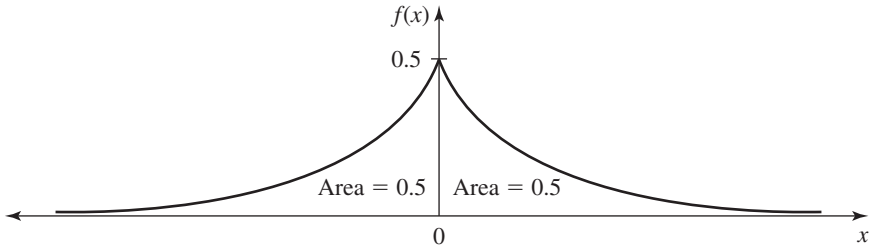


FIGURE 8.6
Double-exponential density.

composition. First generate U_1 and U_2 as IID U(0, 1). If $U_1 \leq 0.5$, return $X = \ln U_2$. On the other hand, if $U_1 > 0.5$, return $X = -\ln U_2$. Note that we are essentially generating an exponential random variate with mean 1 and then changing its sign with probability 0.5. Alternatively, we are generating from the left half of the density in Fig. 8.6 with probability equal to the corresponding area (0.5) and from the right half with probability 0.5.

Note that in Example 8.3, step 2 of the general composition algorithm was accomplished by means of the inverse-transform method for exponential random variates; this illustrates how different general approaches for generating random variates might be combined. Also, we see that *two* random numbers are required to generate a single X in this example; in general, we shall need *at least two* random numbers to use the composition method. (The reader may find it interesting to compare Example 8.3 with the inverse-transform method for generating a double-exponential random variate; see Prob. 8.6.)

In Example 8.3 we obtained the representation for f by dividing the area below the density with a vertical line, namely, the ordinate axis. In the following example, we make a horizontal division instead.

EXAMPLE 8.4. For $0 < a < 1$, the *right-trapezoidal distribution* has density

$$f(x) = \begin{cases} a + 2(1 - a)x & \text{if } 0 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

(see Fig. 8.7). As suggested by the dashed lines, we can think of dividing the area under f into a rectangle having area a and a right triangle with area $1 - a$. Now $f(x)$ can be decomposed as

$$f(x) = aI_{[0,1]}(x) + (1 - a)2xI_{[0,1]}(x)$$

so that $f_1(x) = I_{[0,1]}(x)$, which is simply the U(0, 1) density, and $f_2(x) = 2xI_{[0,1]}(x)$ is a right-triangular density. Clearly, $p_1 = a$ and $p_2 = 1 - a$. The composition method thus calls for generating $U_1 \sim$ U(0, 1) and checking whether $U_1 \leq a$. If so, generate an independent $U_2 \sim$ U(0, 1), and return $X = U_2$. If $U_1 > a$, however, we must generate from the right-triangular distribution. This can be accomplished either by generating $U_2 \sim$ U(0, 1) and returning $X = \sqrt{U_2}$, or by generating U_2 and U_3 distributed as IID U(0, 1) and returning $X = \max\{U_2, U_3\}$ (see Prob. 8.7). Since the time to take a square root is probably greater than that required to generate an extra U(0, 1) random variate *and* to perform a comparison, the latter method would appear to be a faster way of generating an X with density f_2.

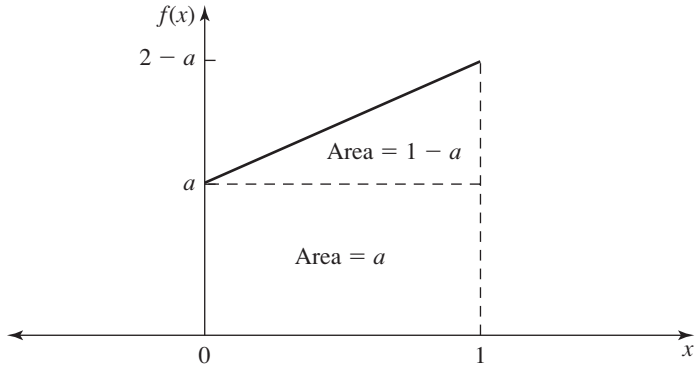


FIGURE 8.7
Right-trapezoidal density.

Again, the reader is encouraged to develop the inverse-transform method for generating a random variate from the right-trapezoidal distribution in Example 8.4. Note that especially if a is large, the composition method will be faster than the inverse transform, since the latter *always* requires that a square root be taken, while it is quite likely (with probability a) that the former will simply return $X = U_2 \sim$ U(0, 1). This increase in speed must be played off by the analyst against the possible disadvantage of having to generate two or three random numbers to obtain one value of X. Trapezoidal distributions like that in Example 8.4 play an important role in the efficient methods developed by Schmeiser and Lal (1980) for generating gamma random variates and for beta generation in Schmeiser and Babu (1980).

Composition methods (also called "mixture" methods) are further analyzed by Peterson and Kronmal (1982), who show as well that many specific variate-generation methods can actually be expressed as a composition of some sort. An interesting technique that is related closely to composition, the *acceptance-complement method*, was proposed by Kronmal and Peterson (1981, 1982); Devroye (1986, pp. 75–81) further discusses this and associated methods.

8.2.3 Convolution

For several important distributions, the desired random variable X can be expressed as a sum of other random variables that are IID and can be generated more readily than direct generation of X. We assume that there are IID random variables Y_1, $Y_2, \ldots, Y_m$ (for fixed m) such that $Y_1 + Y_2 + \cdots + Y_m$ has the same distribution as X; hence we write

$$X = Y_1 + Y_2 + \cdots + Y_m$$

The name of this method, *convolution*, comes from terminology in stochastic processes, where the distribution of X is called the *m-fold convolution* of the distribution

of a Y_j. The reader should take care not to confuse this situation with the method of composition. Here we assume that the *random variable* X can be represented as a sum of other *random variables*, whereas the assumption behind the method of composition is that the *distribution function* of X is a (weighted) sum of other *distribution functions*; the two situations are fundamentally different.

The algorithm for generating the desired random variate X is quite intuitive (let F be the distribution function of X and G be the distribution function of a Y_j):

1. Generate $Y_1, Y_2, \ldots, Y_m$ IID each with distribution function G.
2. Return $X = Y_1 + Y_2 + \cdots + Y_m$.

To demonstrate the validity of this algorithm, recall that we assumed that X and $Y_1 + Y_2 + \cdots + Y_m$ have the same distribution function, namely, F. Thus,

$$P(X \le x) = P(Y_1 + Y_2 + \cdots + Y_m \le x) = F(x)$$

EXAMPLE 8.5. The m-Erlang random variable X with mean β can be defined as the sum of m IID exponential random variables with common mean β/m. Thus, to generate X, we can first generate $Y_1, Y_2, \ldots, Y_m$ as IID exponential with mean β/m (see Example 8.1), then return $X = Y_1 + Y_2 + \cdots + Y_m$. (See Sec. 8.3.3 for an improvement in efficiency of this algorithm.)

The convolution method, when it can be used, is very simple, provided that we can generate the required Y_j's easily. However, depending on the particular parameters of the distribution of X, it may not be the most efficient way. For example, to generate an m-Erlang random variate by the convolution method (as in Example 8.5) when m is large could be very slow. In this case it would be better to recall that the m-Erlang distribution is a special case of the gamma distribution (see Sec. 6.2.2) and to use a general method for generating gamma random variates (see Sec. 8.3.4). See also Devroye (1988).

Convolution is really an example of a more general idea, that of transforming some intermediate random variates into a final variate that has the desired distribution; the transformation with convolution is just adding, and the intermediate variates are IID. There are many other ways to transform intermediate variates, some of which are discussed in Sec. 8.2.6, as well as in Secs. 8.3 and 8.4.

8.2.4 Acceptance-Rejection

The three approaches for generating random variates discussed so far (inverse transform, composition, and convolution) might be called *direct* in the sense that they deal directly with the distribution or random variable desired. The *acceptance-rejection method* is less direct in its approach and can be useful when the direct methods fail or are inefficient. Our discussion is for the continuous case, where we want to generate X having distribution function F and density f; the discrete case is exactly analogous and is treated in Prob. 8.9. The underlying idea dates back to at least von Neumann (1951).

The acceptance-rejection method requires that we specify a function t, called the *majorizing function*, such that $t(x) \geq f(x)$ for all x. Now t will not, in general, be a density since

$$c = \int_{-\infty}^{\infty} t(x)\, dx \geq \int_{-\infty}^{\infty} f(x)\, dx = 1$$

but the function $r(x) = t(x)/c$ clearly *is* a density. (We assume that t is such that $c < \infty$.) We must be able to generate (easily and quickly, we hope) a random variate Y having density r. The general algorithm follows:

1. Generate Y having density r.
2. Generate $U \sim \text{U}(0, 1)$, independent of Y.
3. If $U \leq f(Y)/t(Y)$, return $X = Y$. Otherwise, go back to step 1 and try again.

The algorithm continues looping back to step 1 until finally we generate a (Y, U) pair in steps 1 and 2 for which $U \leq f(Y)/t(Y)$, when we "accept" the value Y for X. Since demonstrating the validity of this algorithm is more complicated than for the three previous methods, we refer the reader to App. 8A for a proof.

EXAMPLE 8.6. The beta(4, 3) distribution (on the unit interval) has density

$$f(x) = \begin{cases} 60x^3(1 - x)^2 & \text{if } 0 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

[Since the distribution function $F(x)$ is a sixth-degree polynomial, the inverse-transform approach would not be simple, involving numerical methods to find polynomial roots.] By standard differential calculus, i.e., setting $df/dx = 0$, we see that the maximum value of $f(x)$ occurs at $x = 0.6$, where $f(0.6) = 2.0736$ (exactly). Thus, if we define

$$t(x) = \begin{cases} 2.0736 & \text{if } 0 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

then t majorizes f. Next, $c = \int_0^1 2.0736\, dx = 2.0736$, so that $r(x)$ is just the U(0, 1) density. The functions f, t, and r are shown in Fig. 8.8. The algorithm first generates Y and U as IID U(0, 1) random variates in steps 1 and 2; then in step 3 we check whether

$$U \leq \frac{60\, Y^3(1 - Y)^2}{2.0736}$$

If so, we return $X = Y$; otherwise, we reject Y and go back to step 1.

Note that in the preceding example, X is bounded on an interval (the unit interval in this case), and so we were able to choose t to be constant over this interval, which in turn led to r's being a uniform density. The acceptance-rejection method is often stated *only* for such bounded random variables X and *only* for this uniform choice of r; our treatment is more general.

The acceptance-rejection algorithm above seems curious to say the least, and the proof of its validity in App. 8A adds little insight. There is, however, a natural intuition to the method. Figure 8.9 presents again the $f(x)$ and $t(x)$ curves from Example 8.6, and in addition shows the algorithm in action. We generated 50 X's from the beta(4, 3) distribution by acceptance-rejection (using stream 2 of the

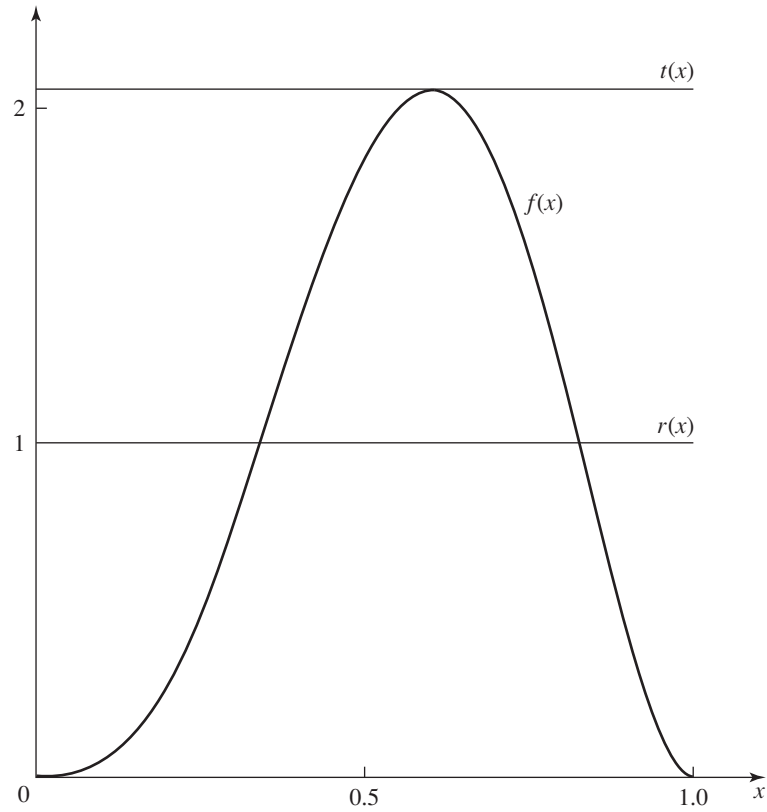


FIGURE 8.8
$f(x)$, $t(x)$, and $r(x)$ for the acceptance-rejection method, beta(4, 3) distribution.

random-number generator in App. 7A), which are marked by crosses on the x axis. On the $t(x)$ curve at the top of the graph we also mark the location of all the Y's generated in step 1 of the algorithm, regardless of whether they ended up being accepted as X's; 50 of these Y's were accepted and made it down to the x axis. The uniformity of the Y's on the $t(x)$ curve is evident, and the higher concentration of the X's on the x axis where $f(x)$ is high is also clear. For those Y's falling in regions where $f(x)$ is low (e.g., x near 0 or 1), $f(Y)/t(Y)$ is small, and as this is the probability of accepting Y as an X, most of such Y's will be rejected. This can be seen in Fig. 8.9 for small (near 0) and large (near 1) values of Y where $f(x)$ is small. On the other hand, Y values where $f(x)$ is high (e.g., near $x = 0.6$) will probably be kept, since $f(Y)/t(Y)$ is nearly 1; thus, most of the Y's around $x = 0.6$ are accepted as X's and make it down to the x axis. In this way, the algorithm "thins out" the Y's from the $r(x)$ density where $t(x)$ is much larger than $f(x)$, but retains most of the Y's where $t(x)$ is only a little higher than $f(x)$. The result is that the concentration of the Y's from $r(x)$ is altered to agree with the desired density $f(x)$.

The principle of acceptance-rejection is quite general, and looking at the above algorithm in a slightly different way clarifies how it can be extended to generation

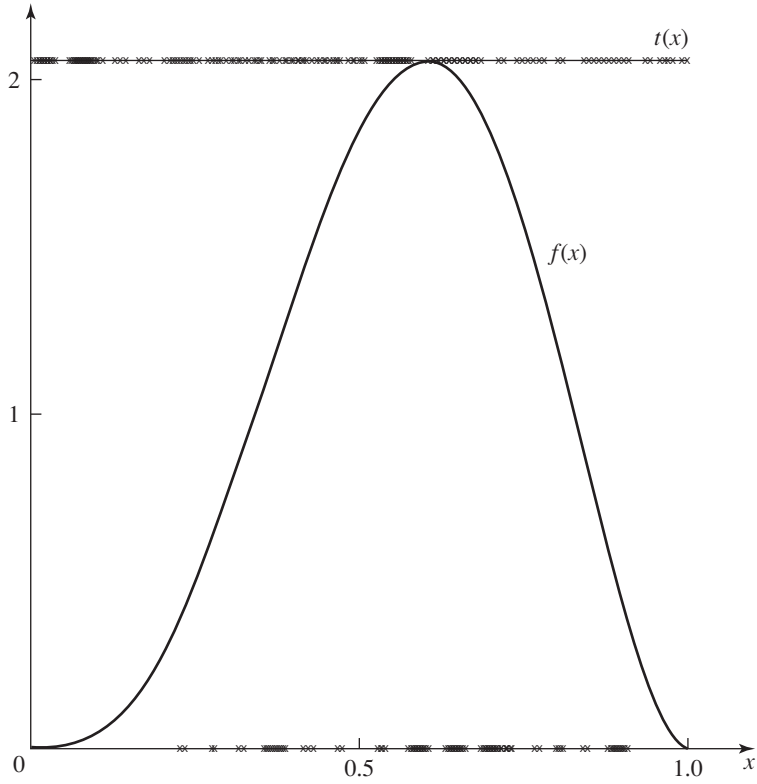


FIGURE 8.9
Sample of 50 X's (on horizontal axis) and the required Y's [on the $t(x)$ line], acceptance-rejection method for beta(4, 3) distribution.

of random points in higher-dimensional spaces; this is important, for example, in Monte Carlo estimation of multiple integrals (see Sec. 13.5). The acceptance condition in step 3 of the algorithm can obviously be restated as $Ut(Y) \leq f(Y)$, which means geometrically that Y will be accepted as an X if the point $(Y, Ut(Y))$ falls under the curve for the density f. Figure 8.10 shows this for the same Y values as in Fig. 8.9, with the dots being the points $(Y, Ut(Y))$ and the 50 accepted values of X again being marked by crosses on the x axis. By accepting the Y values for those $(Y, Ut(Y))$ points falling under the $f(x)$ curve, it is intuitive that the accepted X's will be more dense on the x axis where $f(x)$ is high, since it is more likely that the uniformly distributed dots will be under $f(x)$ there. While in this particular example the rectangular nature of the region under $t(x)$ makes the uniformity of the points $(Y, Ut(Y))$ clear, the same is true for regions of any shape, and in any dimension. The challenge is to find a way of efficiently generating points uniformly in an arbitrary nonrectangular region; Smith (1984) discusses this, and proposes a more efficient alternative to acceptance-rejection in high-dimensional spaces.

Although acceptance-rejection generates a value of X with the desired distribution regardless of the choice of the majorizing function t, this choice will play an

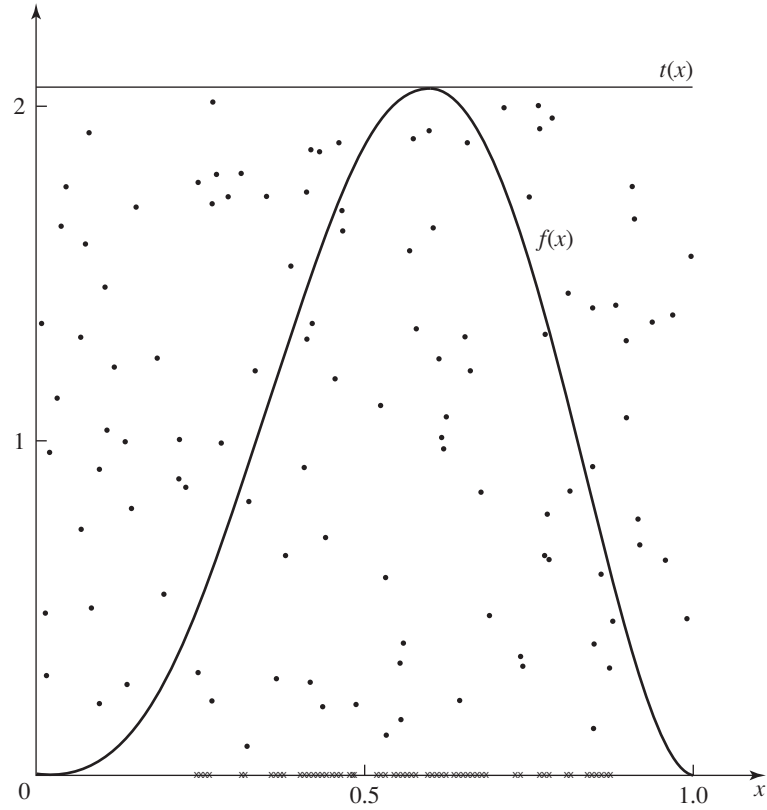


FIGURE 8.10
Sample of 50 X's (on horizontal axis) and the required (Y, $Ut(Y)$) pairs, acceptance-rejection method for beta(4, 3) distribution.

important role in its efficiency in two ways. First, since step 1 requires generating Y with density $t(x)/c$, we want to choose t so that this can be accomplished rapidly. (The uniform t chosen in Example 8.6 certainly satisfies this wish.) Second, we hope that the probability of rejection in step 3 can be made small, since we have to start all over if this rejection occurs. In App. 8A we show that on any given iteration through the algorithm, the probability of acceptance in step 3 is $1/c$; we therefore would like to choose t so that c is small. Thus, we want to find a t that fits closely above f, bringing c closer to 1, its lower bound. Intuitively, a t that is only a little above f leads to a density r that will be close to f, so that the Y values generated from r in step 1 are from a distribution that is almost correct, and so we should accept most of them. (From this standpoint, then, we see that the uniform choice of t in Example 8.6 might not be so wise after all, since it does not fit down on top of f very snugly. Since $c = 2.0736$, the probability of acceptance is only about 0.48, lower than we might like.) These two goals, ease of generation from $t(x)/c$ and a small value of c, may well conflict with each other, so the choice of t is by no means obvious and deserves care. Considerable research has been aimed at identifying

good choices for t for a given distribution; see, for example, Ahrens and Dieter (1972, 1974), Atkinson (1979b), Atkinson and Whittaker (1976), Schmeiser (1980a, 1980b), Schmeiser and Babu (1980), Schmeiser and Lal (1980), Schmeiser and Shalaby (1980), and Tadikamalla (1978). One popular method of finding a suitable t is *first* to specify $r(x)$ to be some common density, e.g., a normal or double exponential, then find the smallest c such that $t(x) = cr(x) \geq f(x)$ for all x.

EXAMPLE 8.7. Consider once again the beta(4, 3) distribution from Example 8.6, but now with a more elaborate majorizing function in an attempt to raise the probability of acceptance without unduly burdening the generation of Y's from $r(x)$; we do this along the lines of Schmeiser and Shalaby (1980). For this density, there are two *inflection points* [i.e., values of x above which $f(x)$ switches from convex to concave, or vice versa], which can be found by solving $f''(x) = 0$ for those values of x between 0 and 1; the solutions are $x = 0.36$ and $x = 0.84$ (to two decimals). Checking the signs of $f''(x)$ on the three regions of [0, 1] created by these two inflection points, we find that $f(x)$ is convex on [0, 0.36], concave on [0.36, 0.84], and again convex on [0.84, 1]. By the definition of convexity, a line from the point (0, 0) to the point (0.36, f(0.36)) will lie above $f(x)$; similarly, the line connecting (0.84, f(0.84)) with the point (1, 0) will be above $f(x)$. Over the concave region, we simply place a horizontal line at height 2.0736, the maximum of $f(x)$. This leads to the piecewise-linear majorizing function $t(x)$ shown in Fig. 8.11; adding up the areas of the

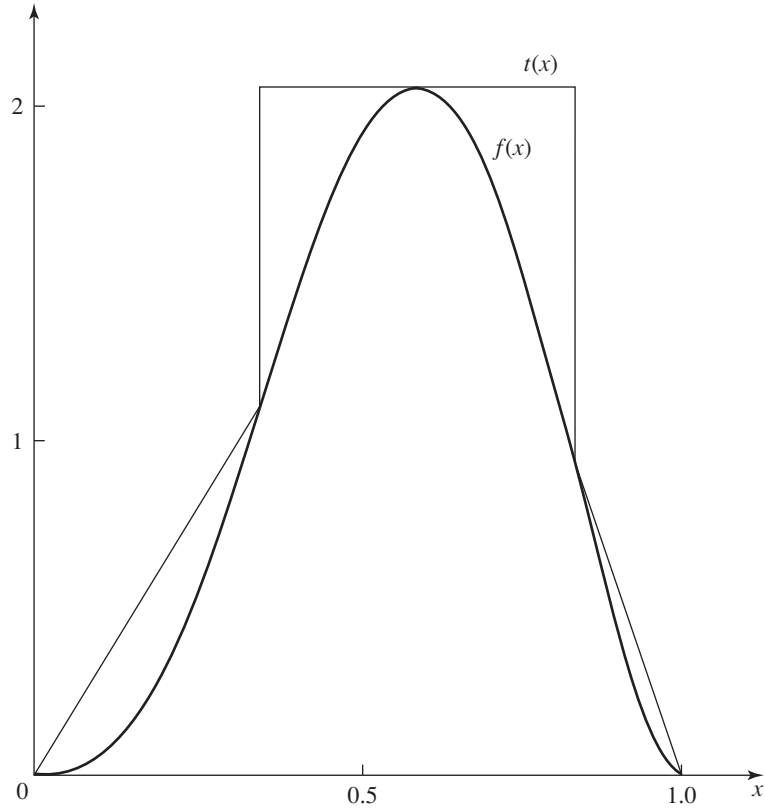


FIGURE 8.11
$f(x)$ and piecewise-linear majorizing function $t(x)$, acceptance-rejection method for beta(4, 3) distribution.

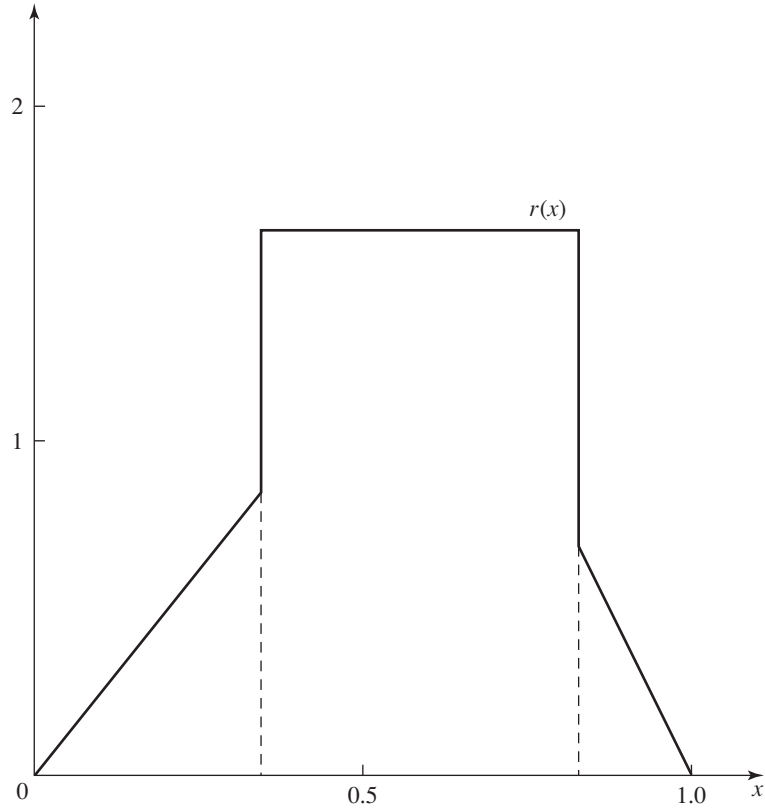


FIGURE 8.12
$r(x)$ corresponding to the piecewise-linear majorizing function $t(x)$ in Fig. 8.11.

two triangles and the rectangle, we get $c = 1.28$, so that the probability of acceptance on a given pass through the algorithm is 0.78, being considerably better than the 0.48 acceptance probability when using the simple uniform majorizing function in Example 8.6. However, it now becomes more difficult to generate values from the density $r(x)$, which is plotted in Fig. 8.12; this is a typical tradeoff in specifying a majorizing function. As suggested in Fig. 8.12, generating from $r(x)$ can be done using composition, dividing the area under $r(x)$ into three regions, each corresponding to a density from which generation is easy; see Prob. 8.16. Thus, we would really be combining three different generation techniques here: inverse transform [for the component densities of $r(x)$], composition [for $r(x)$], and finally acceptance-rejection [for $f(x)$]. In comparison with Example 8.6, whether the higher acceptance probability justifies the increased work to generate a Y is not clear, and may depend on several factors, such as the particular parameters, code efficiency, as well as the programming language, compiler, and hardware.

There have been many variations of and modifications to the general acceptance-rejection method, mostly to improve speed. For example, Hörmann and Derflinger (1996) give a version for generating from discrete distributions (including those with infinite tails), which uses a continuous majorizing function and avoids generating a separate random number to decide between acceptance and rejection.

8.2.5 Ratio of Uniforms

Let $f(x)$ be the density function corresponding to a continuous random variable X, from which we would like to generate random variates. The *ratio-of-uniforms method*, which is due to Kinderman and Monahan (1977), is based on a curious relationship among the random variables U, V, and V/U. Let p be a positive real number. If (U, V) is uniformly distributed over the set

$$S = \left\{ (u, v): 0 \le u \le \sqrt{pf\left(\frac{v}{u}\right)} \right\}$$

then V/U has density function f, which we will now show. The joint density function of U and V is given by

$$f_{U,V}(u, v) = \frac{1}{s} \qquad \text{for } (u, v) \in S$$

where s is the area of the set S. Let $Y = U$ and $Z = V/U$. The Jacobian, J, of this transformation is given by the following determinant:

$$J = \begin{vmatrix} \dfrac{\partial u}{\partial y} & \dfrac{\partial u}{\partial z} \\ \dfrac{\partial v}{\partial y} & \dfrac{\partial v}{\partial z} \end{vmatrix} = \begin{vmatrix} 1 & 0 \\ z & y \end{vmatrix} = y$$

Therefore, the joint distribution of Y and Z is [see, e.g., DeGroot (1975, pp. 133–136)]

$$f_{Y,Z}(y, z) = |J| f_{U,V}(u, v) = \frac{y}{s} \qquad \text{for } 0 \le y \le \sqrt{pf(z)} \text{ and } 0 < z < \infty$$

so that the density function of Z is

$$f_Z(z) = \int_0^{\sqrt{pf(z)}} f_{Y,Z}(y, z)\, dy = \int_0^{\sqrt{pf(z)}} \frac{y}{s}\, dy = \frac{p}{2s} f(z)$$

Since $f_Z(z)$ and $f(z)$ must both integrate to 1, it follows that $s = p/2$ and that $Z = V/U$ has density $f(x)$ as desired.

To generate (u, v) uniformly in S, we may choose a majorizing region T that contains S, generate a point (u, v) uniformly in T, and accept this point if

$$u^2 \le pf\left(\frac{v}{u}\right)$$

Otherwise, we generate a new point in T and try again, etc. Hopefully, it should be easy to generate a point uniformly in the region T.

The boundary of the acceptance region S is defined parametrically by the following equations (see Prob. 8.19):

$$u(z) = \sqrt{pf(z)} \quad \text{and} \quad v(z) = z\sqrt{pf(z)} \tag{8.1}$$

If $f(x)$ and $x^2 f(x)$ are bounded, then a good choice of T is the rectangle

$$T = \{(u, v) : 0 \le u \le u^* \text{ and } v_* \le v \le v^*\}$$

where

$$u^* = \sup_z u(z) = \sup_z \sqrt{pf(z)}$$
$$v_* = \inf_z v(z) = \inf_z z\sqrt{pf(z)}$$
$$v^* = \sup_z v(z) = \sup_z z\sqrt{pf(z)}$$

[The supremum (sup) of the set (0, 1) is 1, but the maximum doesn't exist. The definition of the infimum (inf) is analogous.] With this choice of the majorizing region T, a formal statement of the ratio-of-uniforms method is as follows:

1. Generate $U \sim \mathrm{U}(0, u^*)$ and $V \sim \mathrm{U}(v_*, v^*)$ independently.
2. Set $Z = V/U$.
3. If $U^2 \le pf(Z)$, then return Z. Otherwise, go back to step 1.

If t is the area of the region T, then the probability of accepting a particular Z is

$$\frac{s}{t} = \frac{p/2}{u^*(v^* - v_*)}$$

and the mean number of passes through steps 1 through 3 until a Z is accepted is the inverse of the above ratio.

EXAMPLE 8.8. Suppose that $f(x) = 3x^2$ for $0 \le x \le 1$ [a beta(3, 1) distribution (see Sec. 6.2.2)] and $p = 1/3$. Then

$$S = \left\{(u, v) : 0 \le u \le \frac{v}{u}, 0 \le \frac{v}{u} \le 1\right\}$$
$$s = \frac{1}{6}$$
$$u^* = 1, \quad v_* = 0, \quad v^* = 1$$

We generated 2000 points (u, v) uniformly in the square T (with area $t = 1$), and 330 of these points satisfied the stopping rule stated in the definition of S. Note that $330/2000 = 0.165$, which is approximately equal to $s/t = \frac{1}{6}/1 = 0.167$. A plot of these 330 accepted points is given in Fig. 8.13. The sample mean and sample variance of the 330 selected points were 0.763 and 0.0395, respectively, while the true mean and variance of the beta(3, 1) distribution are 0.750 and 0.0375. Note that the acceptance region S is bounded above and below by the curves $v = u$ and $v = u^2$, respectively (see Prob. 8.20).

The rectangle T had a probability of acceptance of $1/6$ in Example 8.8. We could choose a majorizing region T that is closer in shape to the acceptance region S so that the ratio s/t is brought closer to 1 (see Prob. 8.21). However, this gain in efficiency has to be traded off with the potentially greater difficulty of generating a point uniformly in a more complicated majorizing region. Cheng and Feast (1979) give a fast algorithm for generating from a gamma distribution, where T is a parallelogram. Leydold (2000) develops fast ratio-of-uniforms algorithms that use polygonal majorizing regions and are applicable to a large class of distributions.

Stadlober (1990) shows that the ratio-of-uniforms method is, in fact, an acceptance-rejection method. He also extends the ratio-of-uniforms method to

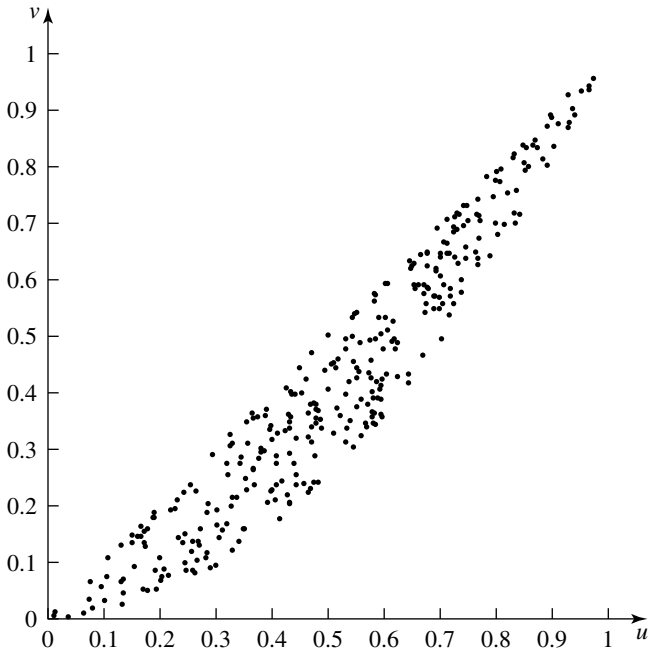


FIGURE 8.13
Accepted points for the ratio-of-uniforms method.

discrete distributions, while Wakefield et al. (1991) and Stefănescu and Văduva (1987) consider multivariate distributions.

8.2.6 Special Properties

Although most methods for generating random variates can be classified into one of the five approaches discussed so far in Sec. 8.2, some techniques simply rely on some *special property* of the desired distribution function F or the random variable X. Frequently, the special property will take the form of representing X in terms of other random variables that are more easily generated; in this sense the method of convolution is a "special" special property. The following four examples are based on normal-theory random variables (see Secs. 8.3 and 8.4 for other examples that have nothing to do with the normal distribution).

EXAMPLE 8.9. If $Y \sim \text{N}(0, 1)$ (the standard normal distribution), then Y^2 has a chi-square distribution with 1 df. (We write $X \sim \chi^2_k$ to mean that X has a chi-square distribution with k df.) Thus, to generate $X \sim \chi^2_1$, generate $Y \sim \text{N}(0, 1)$ (see Sec. 8.3.6), and return $X = Y^2$.

EXAMPLE 8.10. If $Z_1, Z_2, \ldots, Z_k$ are IID χ^2_1 random variables, then $X = Z_1 + Z_2 + \cdots + Z_k \sim \chi^2_k$. Thus, to generate $X \sim \chi^2_k$, first generate $Y_1, Y_2, \ldots, Y_k$ as

IID N(0, 1) random variates, and then return $X = Y_1^2 + Y_2^2 + \cdots + Y_k^2$ (see Example 8.9). Since for large k this may be quite slow, we might want to exploit the fact that the χ_k^2 distribution is a gamma distribution with shape parameter $\alpha = k/2$ and scale parameter $\beta = 2$. Then X can be obtained directly from the gamma-generation methods discussed in Sec. 8.3.4.

EXAMPLE 8.11. If $Y \sim \text{N}(0, 1)$, $Z \sim \chi_k^2$, and Y and Z are independent, then $X = Y/\sqrt{Z/k}$ is said to have *Student's t distribution* with k df, which we denote $X \sim t_k$. Thus, to generate $X \sim t_k$, we generate $Y \sim \text{N}(0, 1)$ and $Z \sim \chi_k^2$ independently of Y (see Example 8.10), and return $X = Y/\sqrt{Z/k}$.

EXAMPLE 8.12. If $Z_1 \sim \chi_{k_1}^2$, $Z_2 \sim \chi_{k_2}^2$, and Z_1 and Z_2 are independent, then

$$X = \frac{Z_1/k_1}{Z_2/k_2}$$

is said to have an *F distribution* with (k_1, k_2) df, denoted $X \sim F_{k_1,k_2}$. We thus generate $Z_1 \sim \chi_{k_1}^2$ and $Z_2 \sim \chi_{k_2}^2$ independently, and return $X = (Z_1/k_1)/(Z_2/k_2)$.

For some continuous distributions, it is possible to transform the density function so that it is easy to construct majorizing functions for use with the acceptance-rejection method. In particular, Hörmann (1995) suggested the *transformed density rejection method* for generating random variates from a continuous distribution with density f. The idea is to transform f by a strictly increasing function T so that $T(f(x))$ is concave, in which case we say that f is *T-concave*. [A function g is said to be *concave* if

$$\frac{g(x_1) + g(x_2)}{2} < g\left(\frac{x_1 + x_2}{2}\right)$$

for $x_1 < x_2$.] Since $T(f(x))$ is a concave function, a majorizing function for $T(f(x))$ can be constructed easily as the minimum of several tangents. Then T^{-1} is used to transform the majorizing function back to the original scale. This results in a majorizing function for the density f, and random variates can be generated from f by the acceptance-rejection method. If $T(x) = -1/\sqrt{x}$, then a large number of distributions are T-concave, including the beta, exponential, gamma, lognormal, normal, and Weibull distributions. (For some distributions, there are restrictions on the values of the parameters.) Since transformed density rejection is applicable to a large class of distributions, it is sometimes called a *universal* method [see Hörmann et al. (2004)].

8.3 GENERATING CONTINUOUS RANDOM VARIATES

In this section we discuss particular algorithms for generating random variates from several commonly occurring continuous distributions; Sec. 8.4 contains a similar treatment for discrete random variates. Although there may be several different algorithms for generating from a given distribution, we explicitly present

only one technique in each case and provide references for other algorithms that may be better in some sense, e.g., in terms of speed at the expense of increased setup cost and greater complexity. In deciding which algorithm to present, we have tried to choose those that are simple to describe and implement, and are reasonably efficient as well. We also give only exact (up to machine accuracy) methods, as opposed to approximations. If speed is critically important, however, we urge the reader to pursue the various references given for the desired distribution. For definitions of density functions, mass functions, and distribution functions, see Secs. 6.2.2 and 6.2.3.

8.3.1 Uniform

The distribution function of a U(a, b) random variable is easily inverted by solving $u = F(x)$ for x to obtain, for $0 \le u \le 1$,

$$x = F^{-1}(u) = a + (b - a)u$$

Thus, we can use the inverse-transform method to generate X:

1. Generate $U \sim$ U(0, 1).
2. Return $X = a + (b - a)U$.

If many X values are to be generated, the constant $b - a$ should, of course, be computed beforehand and stored for use in the algorithm.

8.3.2 Exponential

The exponential random variable with mean $\beta > 0$ was considered in Example 8.1, where we derived the following inverse-transform algorithm:

1. Generate $U \sim$ U(0, 1).
2. Return $X = -\beta \ln U$.

[Recall that the U in step 2 would be $1 - U$ instead if we wanted a literal version of $X = F^{-1}(U)$ in order to make the correlation between the X's and U's positive.] This is certainly a simple technique and has all the advantages of the inverse-transform method discussed in Sec. 8.2.1. It is also reasonably fast, with most of the computing time's being taken up in evaluating the logarithm. In the experiments of Ahrens and Dieter (1972), this method was the fastest of the four algorithms considered if programming in FORTRAN, with some 72 percent of the time taken up by the logarithm evaluation. If one is willing to program in a lower-level language, however, there are other methods that avoid the logarithm and are faster, although considerably more complex and involving various amounts of preliminary setup; see von Neumann (1951), Marsaglia (1961), and MacLaren, Marsaglia, and Bray (1964). We refer the interested reader to Ahrens and Dieter (1972) and to Fishman (1978, pp. 402–410) for further discussion.

8.3.3 *m*-Erlang

As discussed in Example 8.5, if X is an m-Erlang random variable with mean β, we can write $X = Y_1 + Y_2 + \cdots + Y_m$, where the Y_i's are IID exponential random variables, each with mean β/m. This led to the convolution algorithm described in Example 8.5. Its efficiency can be improved, however, as follows. If we use the inverse-transform method of Sec. 8.3.2 to generate the exponential Y_i's [$Y_i = (-\beta/m) \ln U_i$, where $U_1, U_2, \ldots, U_m$ are IID U(0, 1) random variates], then

$$X = \sum_{i=1}^{m} Y_i = \sum_{i=1}^{m} \frac{-\beta}{m} \ln U_i = \frac{-\beta}{m} \ln\left(\prod_{i=1}^{m} U_i\right)$$

so that we need to evaluate only one logarithm (rather than m logarithms). Then the statement of the algorithm is as follows:

1. Generate $U_1, U_2, \ldots, U_m$ as IID U(0, 1).

2. Return $X = \dfrac{-\beta}{m} \ln\left(\prod_{i=1}^{m} U_i\right)$.

(Again, one should compute β/m beforehand and store it for repeated use.) This algorithm is really a combination of the composition and inverse-transform methods.

Since we must generate m random numbers and perform m multiplications, the execution time of the algorithm is approximately proportional to m. Therefore, one might look for an alternative method when m is large. Fortunately, the m-Erlang distribution is a special case of the gamma distribution (with shape parameter α equal to the integer m), so that we can use one of the methods for generating gamma random variates here as well (see Sec. 8.3.4 for discussion of gamma generation). The precise threshold for m beyond which one should switch to general gamma generation will depend on the method used for generating a gamma random variate as well as on languages, compilers, and hardware; preliminary experimentation in one's particular situation might prove worthwhile. [For the gamma generator of Sec. 8.3.4 in the case $\alpha > 1$, timing experiments in Cheng (1977) indicate that using his general gamma generator becomes faster than the above m-Erlang algorithm for $m \geq 10$, approximately.] Another potential problem with using the above algorithm, especially for large m, is that $\prod_{i=1}^{m} U_i$ might get close to zero, which could lead to numerical difficulties when its logarithm is taken.

8.3.4 Gamma

General gamma random variates are more complicated to generate than the three types of random variates considered so far in this section, since the distribution function has no simple closed form for which we could try to find an inverse. First note that given $X \sim$ gamma(α, 1), we can obtain, for any $\beta > 0$, a gamma(α, β) random variate X' by letting $X' = \beta X$, so that it is sufficient to restrict attention

to generating from the gamma(α, 1) distribution. Furthermore, recall that the gamma(1, 1) distribution is just the exponential distribution with mean 1, so that we need consider only $0 < \alpha < 1$ and $\alpha > 1$. Since the available algorithms for generating gamma random variates are for the most part valid in only one of these ranges of α, we shall discuss them separately. [Tadikamalla and Johnson (1981) provide a comprehensive review of gamma variate generation methods that were available at that time.]

We first consider the case $0 < \alpha < 1$. (Note that if $\alpha = 0.5$, we have a rescaled χ_1^2 distribution and X can be easily generated using Example 8.9; the algorithm stated below is nevertheless valid for $\alpha = 0.5$.) Atkinson and Pearce (1976) tested three alterative algorithms for this case, and we present one of them, due to Ahrens and Dieter (1974). [The algorithm of Forsythe (1972) was usually the fastest in the comparisons in Atkinson and Pearce (1976), but it is considerably more complicated.] This algorithm, denoted GS in Ahrens and Dieter (1974), is an acceptance-rejection technique, with majorizing function

$$t(x) = \begin{cases} 0 & \text{if } x \le 0 \\ \dfrac{x^{\alpha-1}}{\Gamma(\alpha)} & \text{if } 0 < x \le 1 \\ \dfrac{e^{-x}}{\Gamma(\alpha)} & \text{if } 1 < x \end{cases}$$

Thus, $c = \int_0^\infty t(x)\,dx = b/[\alpha\Gamma(\alpha)]$, where $b = (e + \alpha)/e > 1$, which yields the density $r(x) = t(x)/c$ as

$$r(x) = \begin{cases} 0 & \text{if } x \le 0 \\ \dfrac{\alpha x^{\alpha-1}}{b} & \text{if } 0 < x \le 1 \\ \dfrac{\alpha e^{-x}}{b} & \text{if } 1 < x \end{cases}$$

Generating a random variate Y with density $r(x)$ can be done by the inverse-transform method; the distribution function corresponding to r is

$$R(x) = \int_0^x r(y)\,dy = \begin{cases} \dfrac{x^\alpha}{b} & \text{if } 0 \le x \le 1 \\ 1 - \dfrac{\alpha e^{-x}}{b} & \text{if } 1 < x \end{cases}$$

which can be inverted to obtain

$$R^{-1}(u) = \begin{cases} (bu)^{1/\alpha} & \text{if } u \le \dfrac{1}{b} \\ -\ln \dfrac{b(1-u)}{\alpha} & \text{otherwise} \end{cases}$$

Thus, to generate Y with density r, we first generate $U_1 \sim \mathrm{U}(0, 1)$. If $U_1 \le 1/b$, we set $Y = (bU_1)^{1/\alpha}$; in this case, $Y \le 1$. Otherwise, if $U_1 > 1/b$, set $Y = -\ln [b(1 - U_1)/\alpha]$, which will be greater than 1. Noting that

$$\frac{f(Y)}{t(Y)} = \begin{cases} e^{-Y} & \text{if } 0 \le Y \le 1 \\ Y^{\alpha-1} & \text{if } 1 < Y \end{cases}$$

we obtain the final algorithm [$b = (e + \alpha)/e$ must be computed beforehand]:

1. Generate $U_1 \sim \mathrm{U}(0, 1)$, and let $P = bU_1$. If $P > 1$, go to step 3. Otherwise, proceed to step 2.
2. Let $Y = P^{1/\alpha}$, and generate $U_2 \sim \mathrm{U}(0, 1)$. If $U_2 \le e^{-Y}$, return $X = Y$. Otherwise, go back to step 1.
3. Let $Y = -\ln [(b - P)/\alpha]$ and generate $U_2 \sim \mathrm{U}(0, 1)$. If $U_2 \le Y^{\alpha-1}$, return $X = Y$. Otherwise, go back to step 1.

We now consider the case $\alpha > 1$, where there are several good algorithms. In view of timing experiments by Schmeiser and Lal (1980) and Cheng and Feast (1979), we will present a modified acceptance-rejection method due to Cheng (1977), who calls this the GB algorithm. This algorithm has a "capped" execution time; i.e., its execution time is bounded as $\alpha \to \infty$ and in fact appears to become faster as α grows. (The modification of the general acceptance-rejection method consists of adding a faster pretest for acceptance.) To obtain a majorizing function $t(x)$, first let $\lambda = \sqrt{2\alpha - 1}$, $\mu = \alpha^\lambda$, and $c = 4\alpha^\alpha e^{-\alpha}/[\lambda\Gamma(\alpha)]$. Then define $t(x) = cr(x)$, where

$$r(x) = \begin{cases} \dfrac{\lambda \mu x^{\lambda-1}}{(\mu + x^\lambda)^2} & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

The distribution function corresponding to the density $r(x)$ is

$$R(x) = \begin{cases} \dfrac{x^\lambda}{\mu + x^\lambda} & \text{if } x \ge 0 \\ 0 & \text{otherwise} \end{cases}$$

which is easily inverted to obtain

$$R^{-1}(u) = \left(\frac{\mu u}{1 - u}\right)^{1/\lambda} \qquad \text{for } 0 < u < 1$$

To verify that $t(x)$ indeed majorizes $f(x)$, see Cheng (1977). Note that this is an example of obtaining a majorizing function by first specifying a known distribution [$R(x)$ is actually the log-logistic distribution function (see Sec. 8.3.11) with shape parameter λ, scale parameter $\mu^{1/\lambda}$, and location parameter 0] and then rescaling the density $r(x)$ to majorize $f(x)$. Thus, we use the inverse-transform method to generate Y with density r. After adding an advantageous pretest for acceptance and streamlining for computational efficiency, Cheng (1977) recommends the following

algorithm (the prespecified constants are $a = 1/\sqrt{2\alpha - 1}$, $b = \alpha - \ln 4$, $q = \alpha + 1/a$, $\theta = 4.5$, and $d = 1 + \ln \theta$):

1. Generate U_1 and U_2 as IID U(0, 1).
2. Let $V = a \ln [U_1/(1 - U_1)]$, $Y = \alpha e^V$, $Z = U_1^2 U_2$, and $W = b + qV - Y$.
3. If $W + d - \theta Z \geq 0$, return $X = Y$. Otherwise, proceed to step 4.
4. If $W \geq \ln Z$, return $X = Y$. Otherwise, go back to step 1.

Step 3 is the added pretest, which (if passed) avoids computing the logarithm in the regular acceptance-rejection test in step 4. (If step 3 were removed, the algorithm would still be valid and would just be the literal acceptance-rejection method.)

As mentioned above, there are several other good algorithms that could be used when $\alpha > 1$. Schmeiser and Lal (1980) present another acceptance-rejection method with $t(x)$ piecewise linear in the "body" of $f(x)$ and exponential in the tails; their algorithm was roughly twice as fast as the one we chose to present above, for α ranging from 1.0001 through 1000. However, their algorithm is more complicated and requires additional time to set up the necessary constants for a given value of α. This is typical of the tradeoffs the analyst must consider in choosing among alternative variate-generation algorithms.

Finally, we consider direct use of the inverse-transform method to generate gamma random variates. Since neither the gamma distribution function nor its inverse has a simple closed form, we must resort to numerical methods. Best and Roberts (1975) give a numerical procedure for inverting the distribution function of a chi-square random variable with degrees of freedom that need not be an integer, so is applicable for gamma generation for any $\alpha > 0$. [If $Y \sim \chi^2_\nu$ where $\nu > 0$ need not be an integer, then $Y \sim$ gamma($\nu/2$, 2). If we want $X \sim$ gamma(α, 1), first generate $Y \sim \chi^2_{2\alpha}$, and then return $X = Y/2$.] An IMSL routine [Rogue Wave (2013)] is available to invert the chi-square distribution function. Press et al. (2007, sec. 6.2) give C codes to evaluate the chi-square distribution function (a reparameterization of what's known as the *incomplete gamma function*), which would then have to be numerically inverted by a root-finding algorithm, which they discuss in their chap. 9.

8.3.5 Weibull

The Weibull distribution function is easily inverted to obtain

$$F^{-1}(u) = \beta[-\ln(1 - u)]^{1/\alpha}$$

which leads to the following inverse-transform algorithm:

1. Generate $U \sim$ U(0, 1).
2. Return $X = \beta(-\ln U)^{1/\alpha}$.

Again we are exploiting the fact that U and $1 - U$ have the same U(0, 1) distribution, so that in step 2, U should be replaced by $1 - U$ if the literal inverse-transform method is desired. This algorithm can also be justified by noting that if Y has

an exponential distribution with mean β^α, then $Y^{1/\alpha} \sim$ Weibull(α, β); see Sec. 6.2.2.

8.3.6 Normal

First note that given $X \sim$ N(0, 1), we can obtain $X' \sim$ N(μ, σ^2) by setting $X' = \mu + \sigma X$, so that we can restrict attention to generating standard normal random variates. Efficiency is important, since the normal density has often been used to provide majorizing functions for acceptance-rejection generation of random variates from other distributions, e.g., Ahrens and Dieter's (1974) gamma and beta generators. Normal random variates can also be transformed directly into random variates from other distributions, e.g., the lognormal. Also, statisticians seeking to estimate empirically, in a Monte Carlo study, the null distribution of a test statistic for normality will need an efficient source of normal random variates. [See, for example, Filliben (1975), Lilliefors (1967), or Shapiro and Wilk (1965).]

One of the early methods for generating N(0, 1) random variates, due to Box and Muller (1958), is evidently still in use despite the availability of much faster algorithms. It does have the advantage, however, of maintaining a one-to-one correspondence between the random numbers used and the N(0, 1) random variates produced; it may thus prove useful for maintaining synchronization in the use of common random numbers or antithetic variates as a variance-reduction technique (see Secs. 11.2 and 11.3). The method simply says to generate U_1 and U_2 as IID U(0, 1), then set $X_1 = \sqrt{-2 \ln U_1} \cos 2\pi U_2$ and $X_2 = \sqrt{-2 \ln U_1} \sin 2\pi U_2$. Then X_1 and X_2 are IID N(0, 1) random variates. Since we obtain the desired random variates in pairs, we could, on odd-numbered calls to the subprogram, actually compute X_1 and X_2 as just described, but return only X_1, saving X_2 for immediate return on the next (even-numbered) call. Thus, we use *two* random numbers to produce *two* N(0, 1) random variates. While this method is valid in principle, i.e., if U_1 and U_2 are truly IID U(0, 1) random variables, there is a serious difficulty if U_1 and U_2 are actually adjacent random numbers produced by a linear congruential generator (see Sec. 7.2), as they might be in practice. Due to the fact that U_2 would depend on U_1 according to the recursion in Eq. (7.1) in Sec. 7.2, it can be shown that the generated variates X_1 and X_2 must fall on a spiral in (X_1, X_2) space, rather than being truly independently normally distributed; see, for example, Bratley, Fox, and Schrage (1987, pp. 223–224). Thus, the Box-Muller method should not be used with a single stream of a linear congruential generator; it might be possible to use separate streams or a composite generator instead, e.g., the combined multiple recursive generator in App. 7B, but one of the methods described below for normal variate generation should probably be used instead.

An improvement to the Box and Muller method, which eliminates the trigonometric calculations and was described in Marsaglia and Bray (1964), has become known as the *polar method*. It relies on a special property of the normal distribution and was found by Atkinson and Pearce (1976) to be between 9 and 31 percent faster in FORTRAN programming than the Box and Muller method, depending on the machine used. [Ahrens and Dieter (1972) experienced a 27 percent reduction

in time.] The polar method, which also generates N(0, 1) random variates in pairs, is as follows:

1. Generate U_1 and U_2 as IID U(0, 1); let $V_i = 2U_i - 1$ for $i = 1, 2$; and let $W = V_1^2 + V_2^2$.
2. If $W > 1$, go back to step 1. Otherwise, let $Y = \sqrt{(-2 \ln W)/W}$, $X_1 = V_1 Y$, and $X_2 = V_2 Y$. Then X_1 and X_2 are IID N(0, 1) random variates.

Since a "rejection" of U_1 and U_2 can occur in step 2 (with probability $1 - \pi/4$, by Prob. 8.12), the polar method will require a random number of U(0, 1) random variates to generate each pair of N(0, 1) random variates. More recently, a very fast algorithm for generating N(0, 1) random variates was developed by Kinderman and Ramage (1976), which is more complicated but required 30 percent less time than the polar method in their FORTRAN experiments.

For direct use of the inverse-transform method in normal generation, one must use a numerical method, since neither the normal distribution function nor its inverse has a simple closed-form expression. Such a method is given by Moro (1995). Also, the IMSL [Rogue Wave (2013)] library has routines to invert the standard normal distribution function.

8.3.7 Lognormal

A special property of the lognormal distribution, namely, that if $Y \sim \text{N}(\mu, \sigma^2)$, then $e^Y \sim \text{LN}(\mu, \sigma^2)$, is used to obtain the following algorithm:

1. Generate $Y \sim \text{N}(\mu, \sigma^2)$.
2. Return $X = e^Y$.

To accomplish step 1, any method discussed in Sec. 8.3.6 for normal generation can be used.

Note that μ and σ^2 are *not* the mean and variance of the LN(μ, σ^2) distribution. In fact, if $X \sim \text{LN}(\mu, \sigma^2)$ and we let $\mu' = E(X)$ and $\sigma'^2 = \text{Var}(X)$, then it turns out that $\mu' = e^{\mu + \sigma^2/2}$ and $\sigma'^2 = e^{2\mu + \sigma^2}(e^{\sigma^2} - 1)$. Thus, if we want to generate a lognormal random variate X with *given* mean $\mu' = E(X)$ and *given* variance $\sigma'^2 = \text{Var}(X)$, we should solve for μ and σ^2 in terms of μ' and σ'^2 first, *before* generating the intermediate normal random variate Y. The formulas are easily obtained as

$$\mu = E(Y) = \ln \frac{\mu'^2}{\sqrt{\mu'^2 + \sigma'^2}}$$

and

$$\sigma^2 = \text{Var}(Y) = \ln\left(1 + \frac{\sigma'^2}{\mu'^2}\right)$$

8.3.8 Beta

First note that we can obtain $X' \sim \text{beta}(\alpha_1, \alpha_2)$ on the interval $[a, b]$ for $a < b$ by setting $X' = a + (b - a)X$, where $X \sim \text{beta}(\alpha_1, \alpha_2)$ on the interval [0, 1], so that it is sufficient to consider only the latter case, which we henceforth call *the* beta(α_1, α_2) distribution.

Some properties of the beta(α_1, α_2) distribution for certain (α_1, α_2) combinations facilitate generating beta random variates. First, if $X \sim$ beta(α_1, α_2), then $1 - X \sim$ beta(α_2, α_1), so that we can readily generate a beta(α_2, α_1) random variate if we can obtain a beta(α_1, α_2) random variate easily. One such situation occurs when either α_1 or α_2 is equal to 1. If $\alpha_2 = 1$, for example, then for $0 \le x \le 1$ we have $f(x) = \alpha_1 x^{\alpha_1 - 1}$, so the distribution function is $F(x) = x^{\alpha_1}$, and we can easily generate $X \sim$ beta(α_1, 1) by the inverse-transform method, i.e., by returning $X = U^{1/\alpha_1}$, for $U \sim$ U(0, 1). Finally, the beta(1, 1) distribution is simply U(0, 1).

A general method for generating a beta(α_1, α_2) random variate for any $\alpha_1 > 0$ and $\alpha_2 > 0$ is a result of the fact that if $Y_1 \sim$ gamma(α_1, 1), $Y_2 \sim$ gamma(α_2, 1), and Y_1 and Y_2 are independent, then $Y_1/(Y_1 + Y_2) \sim$ beta(α_1, α_2). This leads to the following algorithm:

1. Generate $Y_1 \sim$ gamma(α_1, 1) and $Y_2 \sim$ gamma(α_2, 1) independent of Y_1.
2. Return $X = Y_1/(Y_1 + Y_2)$.

Generating the two gamma random variates Y_1 and Y_2 can be done by any appropriate algorithm for gamma generation (see Sec. 8.3.4), so that we must take care to check whether α_1 and α_2 are less than or greater than 1.

This method is quite convenient, in that it is essentially done provided that we have gamma(α, 1) generators for all $\alpha > 0$; its efficiency will, of course, depend on the speed of the chosen gamma generators. There are, however, considerably faster (and more complicated, as usual) algorithms for generating from the beta distribution directly. For $\alpha_1 > 1$ and $\alpha_2 > 1$, Schmeiser and Babu (1980) present a very fast acceptance-rejection method, where the majorizing function is piecewise linear over the center of $f(x)$ and exponential over the tails; a fast acceptance pretest is specified by a piecewise-linear function $b(x)$ that minorizes (i.e., is always below) $f(x)$. If $\alpha_1 < 1$ or $\alpha_2 < 1$ (or both), algorithms for generating beta(α_1, α_2) random variates directly are given by Atkinson and Whittaker (1976, 1979), Cheng (1978), and Jöhnk (1964). Cheng's (1978) method BA is quite simple and is valid as well for any $\alpha_1 > 0$, $\alpha_2 > 0$ combination; the same is true for the algorithms of Atkinson (1979a) and Jöhnk (1964).

The inverse-transform method for generating beta random variates must rely on numerical methods, as was the case for the gamma and normal distributions. Cran, Martin, and Thomas (1977) give such a method with a FORTRAN program, and IMSL [Rogue Wave (2013)] routines are also available. Press et al. (2007, sec. 6.4) give C codes to evaluate the beta distribution function (also known as the *incomplete beta function*), which would then have to be numerically inverted by a root-finding algorithm, which they discuss in their chap. 9.

8.3.9 Pearson Type V

As noted in Sec. 6.2.2, $X \sim$ PT5(α, β) if and only if $1/X \sim$ gamma(α, $1/\beta$), which leads to the following special-property algorithm:

1. Generate $Y \sim$ gamma(α, $1/\beta$).
2. Return $X = 1/Y$.

Any method from Sec. 8.3.4 for gamma generation could be used, taking care to note whether $\alpha < 1$, $\alpha = 1$, or $\alpha > 1$. To use the inverse-transform method, we note from Sec. 6.2.2 that the PT5(α, β) distribution function is $F(x) = 1 - F_G(1/x)$ for $x > 0$, where F_G is the gamma(α, $1/\beta$) distribution function. Setting $F(X) = U$ thus leads to $X = 1/F_G^{-1}(1 - U)$ as the literal inverse-transform method, or to $X = 1/F_G^{-1}(U)$ if we want to exploit the fact that $1 - U$ and U have the same U(0, 1) distribution. In any case, we would generally have to use a numerical method to evaluate F_G^{-1}, as discussed in Sec. 8.3.4.

8.3.10 Pearson Type VI

From Sec. 6.2.2, we note that if $Y_1 \sim$ gamma(α_1, β) and $Y_2 \sim$ gamma(α_2, 1), and Y_1 and Y_2 are independent, then $Y_1/Y_2 \sim$ PT6(α_1, α_2, β); this leads directly to:

1. Generate $Y_1 \sim$ gamma(α_1, β) and $Y_2 \sim$ gamma(α_2, 1) independent of Y_1.
2. Return $X = Y_1/Y_2$.

Any method from Sec. 8.3.4 for gamma generation could be used, checking whether $\alpha < 1$, $\alpha = 1$, or $\alpha > 1$. To use the inverse-transform method, note from Sec. 6.2.2 that the PT6(α_1, α_2, β) distribution function is $F(x) = F_B(x/(x + \beta))$ for $x > 0$, where F_B is the beta(α_1, α_2) distribution function. Setting $F(X) = U$ thus leads to $X = \beta F_B^{-1}(U)/[1 - F_B^{-1}(U)]$, where $F_B^{-1}(U)$ would generally have to be evaluated by a numerical method, as discussed in Sec. 8.3.8.

8.3.11 Log-Logistic

The log-logistic distribution function can be inverted to obtain

$$F^{-1}(u) = \beta\left(\frac{u}{1 - u}\right)^{1/\alpha}$$

which leads to the inverse-transform algorithm:

1. Generate $U \sim$ U(0, 1).
2. Return $X = \beta[U/(1 - U)]^{1/\alpha}$.

8.3.12 Johnson Bounded

$X \sim$ JSB(α_1, α_2, a, b) if and only if $Z = \alpha_1 + \alpha_2 \ln[(X - a)/(b - X)] \sim$ N(0, 1), and we can solve this equation for X in terms of Z to get the following special-property algorithm:

1. Generate $Z \sim$ N(0, 1).
2. Let $Y = \exp[(Z - \alpha_1)/\alpha_2]$.
3. Return $X = (a + bY)/(Y + 1)$.

Any method from Sec. 8.3.6 for standard normal generation can be used to generate Z in step 1.

8.3.13 Johnson Unbounded

$X \sim \text{JSU}(\alpha_1, \alpha_2, \gamma, \beta)$ if and only if

$$Z = \alpha_1 + \alpha_2 \ln\left[\frac{X - \gamma}{\beta} + \sqrt{\left(\frac{X - \gamma}{\beta}\right)^2 + 1}\right] \sim \text{N}(0, 1)$$

and we can solve this equation for X in terms of Z to get the following special-property algorithm:

1. Generate $Z \sim \text{N}(0, 1)$.
2. Let $Y = \exp[(Z - \alpha_1)/\alpha_2]$.
3. Return $X = \gamma + (\beta/2)(Y - 1/Y)$.

Any method from Sec. 8.3.6 for standard normal generation can be used to generate Z in step 1. An alternative statement of the algorithm is $X = \gamma + \beta \sinh[(Z - \alpha_1)/\alpha_2]$ where Z is as in step 1.

8.3.14 Bézier

Random variates from fitted Bézier distributions, as discussed in Sec. 6.9, can be generated by a numerical inverse-transform method given by Wagner and Wilson (1996b), which requires a root-finding algorithm as part of its operation.

8.3.15 Triangular

First notice that if we have $X \sim \text{triang}[0, 1, (m - a)/(b - a)]$, then $X' = a + (b - a)X \sim \text{triang}(a, b, m)$, so we can restrict attention to triang$(0, 1, m)$ random variables, where $0 < m < 1$. (For the limiting cases $m = 0$ or $m = 1$, giving rise to a left or right triangle, see Prob. 8.7.) The distribution function is easily inverted to obtain, for $0 \le u \le 1$,

$$F^{-1}(u) = \begin{cases} \sqrt{mu} & \text{if } 0 \le u \le m \\ 1 - \sqrt{(1 - m)(1 - u)} & \text{if } m < u \le 1 \end{cases}$$

Therefore, we can state the following inverse-transform algorithm for generating $X \sim \text{triang}(0, 1, m)$:

1. Generate $U \sim \text{U}(0, 1)$.
2. If $U \le m$, return $X = \sqrt{mU}$. Otherwise, return $X = 1 - \sqrt{(1 - m)(1 - U)}$.

(Note that if $U > m$ in step 2, we *cannot* replace the $1 - U$ in the formula for X by U. Why?) For an alternative method of generating a triangular random variate (by composition), see Prob. 8.13.

8.3.16 Empirical Distributions

In this section we give algorithms for generating random variates from the continuous empirical distribution functions F and G defined in Sec. 6.2.4. In both cases, the inverse-transform approach can be used.

First suppose that we have the original individual observations, which we use to define the empirical distribution function $F(x)$ given in Sec. 6.2.4 (see also Fig. 6.24). Although an inverse-transform algorithm might at first appear to involve some kind of a search, the fact that the "corners" of F occur precisely at levels $0, 1/(n-1), 2/(n-1), \ldots, (n-2)/(n-1)$, and 1 allows us to avoid an explicit search. We leave it to the reader to verify that the following algorithm *is* the inverse-transform method:

1. Generate $U \sim \text{U}(0, 1)$, let $P = (n-1)U$, and let $I = \lfloor P \rfloor + 1$.
2. Return $X = X_{(I)} + (P - I + 1)(X_{(I+1)} - X_{(I)})$.

Note that the $X_{(i)}$'s must be stored and that storing a separate array containing the values of $X_{(I+1)} - X_{(I)}$ would eliminate a subtraction in step 2. Also, the values of X generated will always be between $X_{(1)}$ and $X_{(n)}$; this limitation is a possible disadvantage of specifying an empirical distribution in this way. The lack of a search makes the marginal execution time of this algorithm essentially independent of n, although large n entails more storage and setup time for sorting the X_i's.

Now suppose that our data are grouped; that is, we have k adjacent intervals $[a_0, a_1), [a_1, a_2), \ldots, [a_{k-1}, a_k]$, and the jth interval contains n_j observations, with $n_1 + n_2 + \cdots + n_k = n$. In this case, we defined an empirical distribution function $G(x)$ in Sec. 6.2.4 (see also Fig. 6.25), and the following inverse-transform algorithm generates a random variate with this distribution:

1. Generate $U \sim \text{U}(0, 1)$.
2. Find the nonnegative integer J $(0 \le J \le k-1)$ such that $G(a_J) \le U < G(a_{J+1})$, and return $X = a_J + [U - G(a_J)](a_{J+1} - a_J)/[G(a_{J+1}) - G(a_J)]$.

Note that the J found in step 2 satisfies $G(a_J) < G(a_{J+1})$, so that no X can be generated in an interval for which $n_j = 0$. (Also, it is clear that $a_0 \le X \le a_k$.) Determining J in step 2 could be done by a straightforward left-to-right search or by a search starting with the value of j for which $G(a_{j+1}) - G(a_j)$ is largest, then next largest, etc. As an alternative that avoids the search entirely (at the expense of extra storage), we could initially define a vector $(m_1, m_2, \ldots, m_n)$ by setting the first n_1 m_i's to 0, the next n_2 m_i's to 1, etc., with the last n_k m_i's being set to $k-1$. (If some n_j is 0, no m_i's are set to $j-1$. For example, if $k \ge 3$ and $n_1 > 0$, $n_2 = 0$, and $n_3 > 0$, the first n_1 m_i's are set to 0 and the *next* n_3 m_i's are set to 2.) Then the value of J in step 2 can be determined by setting $L = \lfloor nU \rfloor + 1$ and letting $J = m_L$. Whether or not this is worthwhile depends on the particular characteristics of the data and on

the importance of any computational speed that might be gained relative to the extra storage and programming effort. Finally, Chen and Asau (1974) give another method for determining J in step 2, based on preliminary calculations that reduce the range of search for a given U; it requires only 10 extra memory locations. (Their treatment is for a discrete empirical distribution function but can also be applied to the present case.)

The empirical/exponential distribution mentioned briefly in Sec. 6.2.4 can also be inverted so that the inverse-transform method can be used; an explicit algorithm is given in Bratley, Fox, and Schrage (1987, p. 151).

8.4 GENERATING DISCRETE RANDOM VARIATES

This section discusses particular algorithms for generating random variates from various discrete distributions that might be useful in a simulation study. As in Sec. 8.3, we usually present for each distribution one algorithm that is fairly simple to implement and reasonably efficient. References will be made to alternative algorithms that might be faster, usually at the expense of greater complexity.

The discrete inverse-transform method, as described in Sec. 8.2.1, can be used for any discrete distribution, whether the range of possible values is finite or (countably) infinite. Many of the algorithms presented in this section *are* the discrete inverse-transform method, although in some cases this fact is very well disguised due to the particular way the required search is performed, which often takes advantage of the special form of the probability mass function. As was the case for continuous random variates, however, the inverse-transform method may not be the most efficient way to generate a random variate from a given distribution.

One other general approach should be mentioned here, which can be used for generating *any* discrete random variate having a *finite* range of values. This is the *alias method*, developed by Walker (1977) and refined by Kronmal and Peterson (1979); it is very general and efficient, but it does require some initial setup as well as extra storage. We discuss the alias method in greater detail in Sec. 8.4.3, but the reader should keep in mind that it is applicable to *any* discrete distribution with a finite range (such as the binomial). For an infinite range, the alias method can be used indirectly in conjunction with the general composition approach (see Sec. 8.2.2); this is also discussed in Sec. 8.4.3.

In addition to the alias method, there are some other general discrete-variate generation ideas; see, for example, Shanthikumar (1985) and Peterson and Kronmal (1983).

A final comment concerns the apparent loss of generality in considering below only distributions that have range $S_n = \{0, 1, 2, \ldots, n\}$ or $S = \{0, 1, 2, \ldots\}$, which may appear to be more restrictive than our original definition of a discrete random variable having general range $T_n = \{x_1, x_2, \ldots, x_n\}$ or $T = \{x_1, x_2, \ldots\}$. However, no generality is actually lost. If we really want a random variate X with mass function $p(x_i)$ and general range T_n (or T), we can first generate a random variate I with range S_{n-1} (or S) such that $P(I = i - 1) = p(x_i)$ for $i = 1, 2, \ldots, n$

(or $i = 1, 2, \ldots$). Then the random variate $X = x_{I+1}$ is returned and has the desired distribution. (Given I, x_{I+1} could be determined from a stored table of the x_i's or from a formula that computes x_i as a function of i.)

8.4.1 Bernoulli

The following algorithm is quite intuitive and is equivalent to the inverse-transform method (if the roles of U and $1 - U$ are reversed):

1. Generate $U \sim \mathrm{U}(0, 1)$.
2. If $U \leq p$, return $X = 1$. Otherwise, return $X = 0$.

8.4.2 Discrete Uniform

Again, the straightforward intuitive algorithm given below is (exactly) the inverse-transform method:

1. Generate $U \sim \mathrm{U}(0, 1)$.
2. Return $X = i + \lfloor (j - i + 1)U \rfloor$.

Note that no search is required. The constant $j - i + 1$ should, of course, be computed ahead of time and stored.

8.4.3 Arbitrary Discrete Distribution

Consider the very general situation in which we have *any* probability mass function $p(0), p(1), p(2), \ldots$ on the nonnegative integers S, and we want to generate a discrete random variate X with the corresponding distribution. The $p(i)$'s could have been specified theoretically by some distributional form or empirically from a data set directly. The case of finite range S_n is included here by setting $p(i) = 0$ for all $i \geq n + 1$. (Note that this formulation includes *every* special discrete distribution form.)

The direct inverse-transform method, for either the finite- or infinite-range case, is as follows (define the empty sum to be 0):

1. Generate $U \sim \mathrm{U}(0, 1)$.
2. Return the nonnegative integer $X = I$ satisfying

$$\sum_{j=0}^{I-1} p(j) \leq U < \sum_{j=0}^{I} p(j)$$

Note that this algorithm will never return a value $X = i$ for which $p(i) = 0$, since the strict inequality between the two summations in step 2 would be impossible. Step 2 does require a search, which may be time consuming. As an alternative, we could initially sort the $p(i)$'s into decreasing order so that the search would be likely to terminate after a smaller number of comparisons; see Prob. 8.2 for an example.

Due to the generality of the present situation, we present three other methods that are useful when the desired random variable has *finite* range S_n. The first of these methods assumes that each $p(i)$ is exactly equal to a q-place decimal; for exposition we take the case $q = 2$, so that $p(i)$ is of the form $0.01k_i$ for some integer $k_i \in \{0, 1, \ldots, 100\}$ ($i = 0, 1, 2, \ldots, n$), and $\sum_{i=0}^{n} k_i = 100$. We initialize a vector $(m_1, m_2, \ldots, m_{100})$ by setting the first k_0 m_j's to 0, the next k_1 m_j's to 1, etc., and the last k_n m_j's to n. (If $k_i = 0$ for some i, no m_j's are set to i.) Then an algorithm for generating the desired random variate X is as follows:

1. Generate $J \sim \text{DU}(1, 100)$.
2. Return $X = m_J$.

(See Sec. 8.4.2 to accomplish step 1.) Note that this method requires 10^q extra storage locations and an array reference in step 2; it *is*, however, the inverse-transform method provided that J is generated by the algorithm in Sec. 8.4.2. If three or four decimal places are needed to specify the $p(i)$'s exactly, the value of l00 in step 1 would be replaced by 1000 or 10,000, respectively, and the storage requirements would also grow by one or two orders of magnitude. Even if the $p(i)$'s are not *exactly* q-place decimals for some small value of q, the analyst might be able to obtain sufficient accuracy by rounding the $p(i)$'s to the nearest hundredth or thousandth; this is an attractive alternative especially when the $p(i)$'s are proportions obtained directly from data, and may not be accurate beyond two or three decimal places anyway. When rounding the $p(i)$'s, however, it is important to remember that they must sum exactly to 1.

The above idea is certainly fast, but it could require large tables if we need high precision in the probabilities. Marsaglia (1963) proposed another kind of table-based algorithm requiring less storage and only slightly more time. For example, consider the distribution

$$p(0) = 0.15, \qquad p(1) = 0.20, \qquad p(2) = 0.37, \qquad p(3) = 0.28$$

Then the idea of the preceding paragraph would require a vector of length 100 to store 15 0s, 20 1s, 37 2s, and 28 3s. Instead, define *two* vectors—one for the "tenths" place and the other for the "hundredths" place. To fill up the tenths vector, look only at the tenths place in the probabilities, and put in that many copies of the associated i (for $i = 0, 1, 2, 3$), so we would take one 0, two 1s, three 2s, and two 3s to get

0 1 1 2 2 2 3 3

Similarly, the hundredths vector is

0 0 0 0 0 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3

corresponding to the hundredths place in the probabilities; thus, there are 28 storage locations in all (as opposed to 100 for the earlier table method). To generate an X, pick the tenths vector with probability equal to one-tenth the sum of the digits in the probabilities' tenths places, i.e., with probability

$$\frac{1 + 2 + 3 + 2}{10} = 0.8$$

and then choose one of the eight members of the tenths vector at random (equiprobably) as the returned X. On the other hand, we choose the hundredths vector with probability equal to $\frac{1}{100}$ of the sum of the digits in the hundredths place in the original probabilities, i.e., with probability

$$\frac{5 + 0 + 7 + 8}{100} = 0.2$$

and then choose one of the 20 entries in the hundredths vector at random to return as the value of X. It is easy to see that this method is valid; for example,

$$\begin{aligned} P(X = 2) &= P(X = 2 \,|\, \text{choose tenths vector})P(\text{choose tenths vector}) \\ &\quad + P(X = 2 \,|\, \text{choose hundredths vector})P(\text{choose hundredths vector}) \\ &= \tfrac{3}{8}(0.8) + \tfrac{7}{20}(0.2) \\ &= 0.37 \end{aligned}$$

as required. The storage advantage of Marsaglia's tables becomes more marked as the number of decimal places in the probabilities increases; in his original example there were three-place decimals, so the direct table method of the preceding paragraph would require 1000 storage locations; the three vectors (tenths, hundredths, and thousandths) in this example required only 91 locations.

The third attractive technique to use when X has range S_n is the alias method mentioned earlier. The method requires that we initially calculate two arrays of length $n + 1$ each, from the given $p(i)$'s. The first array contains what are called the *cutoff values* $F_i \in [0, 1]$ for $i = 0, 1, \ldots, n$, and the second array gives the *aliases* $L_i \in S_n$ for $i = 0, 1, \ldots, n$; two algorithms for computing valid cutoff values and aliases from the $p(i)$'s are given in App. 8B. (The cutoff values and aliases are not unique, and indeed the two algorithms in App. 8B may produce different results for the same distribution; both will result in a valid variate-generation algorithm, however.) Then the alias method is as follows:

1. Generate $I \sim \text{DU}(0, n)$ and $U \sim \text{U}(0, 1)$ independent of I.
2. If $U \le F_I$, return $X = I$. Otherwise, return $X = L_I$.

Thus, step 2 involves a kind of "rejection," but upon rejecting I we need *not* start over but only return I's alias L_I, rather than I itself. The cutoff values are seen to be the probabilities with which we return I rather than its alias. There is only one comparison needed to generate each X, and we need exactly two random numbers for each X if I is generated as in Sec. 8.4.2. (See Prob. 8.17 for a way to accomplish step 1 with only *one* random number.) Although the setup is not complicated, storing the cutoffs and aliases does require $2(n + 1)$ extra storage locations; Kronmal and Peterson (1979) discuss a way to cut the storage in half (see Prob. 8.18). In any case, storage is of order n, which is regarded as the principal weakness of the alias method if n could be very large.

EXAMPLE 8.13. Consider a random variable on $S_3 = \{0, 1, 2, 3\}$ with probability mass function $p(0) = 0.1$, $p(1) = 0.4$, $p(2) = 0.2$, and $p(3) = 0.3$. Applying the first algorithm in App. 8B leads to the following setup:

i	0	1	2	3
$p(i)$	0.1	0.4	0.2	0.3
F_i	0.4	0.0	0.8	0.0
L_i	1	1	3	3

For instance, if step 1 of the algorithm produces $I = 2$, the probability is $F_2 = 0.8$ that we would keep $X = I = 2$, and with probability $1 - F_2 = 0.2$ we would return $X = L_2 = 3$ instead. Thus, since 2 is not the alias of anything else (i.e., none of the other L_i's is equal to 2), the algorithm returns $X = 2$ if and only if $I = 2$ in step 1 and $U \leq 0.8$ in step 2, so that

$$\begin{aligned} P(X = 2) &= P(I = 2 \text{ and } U \leq 0.8) \\ &= P(I = 2)P(U \leq 0.8) \\ &= 0.25 \times 0.8 \\ &= 0.2 \end{aligned}$$

which is equal to $p(2)$, as desired. (The second equality in the above follows since U and I are generated independently.) On the other hand, the algorithm can return $X = 3$ in two different (and mutually exclusive) ways: if $I = 3$, then since $F_3 = 0$ we will always return $X = L_3 = 3$; and if $I = 2$ we will return $X = L_2 = 3$ with probability $1 - F_2 = 0.2$. Thus,

$$\begin{aligned} P(X = 3) &= P(I = 3) + P(I = 2 \text{ and } U > F_2) \\ &= 0.25 + (0.25 \times 0.2) \\ &= 0.3 \end{aligned}$$

which is $p(3)$. The reader is encouraged to verify that the algorithm is correct for $i = 0$ and 1 as well. Figure 8.14 illustrates the method's rationale. Figure 8.14*a* shows bars whose (total) height is $1/(n + 1) = 0.25$, and thus is the probability mass function of I generated in step 1. The shaded areas in the bars represent the probability mass that is moved by the method, and the number in each shaded area is the value L_i that will be returned as X. Thus, if step 1 generates $I = 0$, there is a probability of $1 - F_0 = 0.6$ that this I will be changed into its alias, $L_0 = 1$, for the returned X; the shaded area in the bar above 0 is of height $0.6 \times 0.25 = 0.15$, or 60 percent of that bar. Similarly, the fraction $1 - F_2 = 0.2$ of the 0.25-high bar (resulting in a shaded area of height $0.2 \times 0.25 = 0.05$) above 2 represents the chance that a generated $I = 2$ will be changed into $X = L_2 = 3$. Note that the entire bars above 1 and 3 are shaded, since F_1 and F_3 are both zero; however, the indicated values are their own aliases, so they do not really get moved. Figure 8.14*b* shows the probability mass function of the returned X after the shaded areas (probabilities) are moved to their destination values, and it is seen to equal the desired probabilities $p(i)$.

Although the alias method is limited to discrete random variables with a finite range, it can be used indirectly for discrete distributions with an infinite range, such as the geometric, negative binomial, or Poisson, by combining it with the general

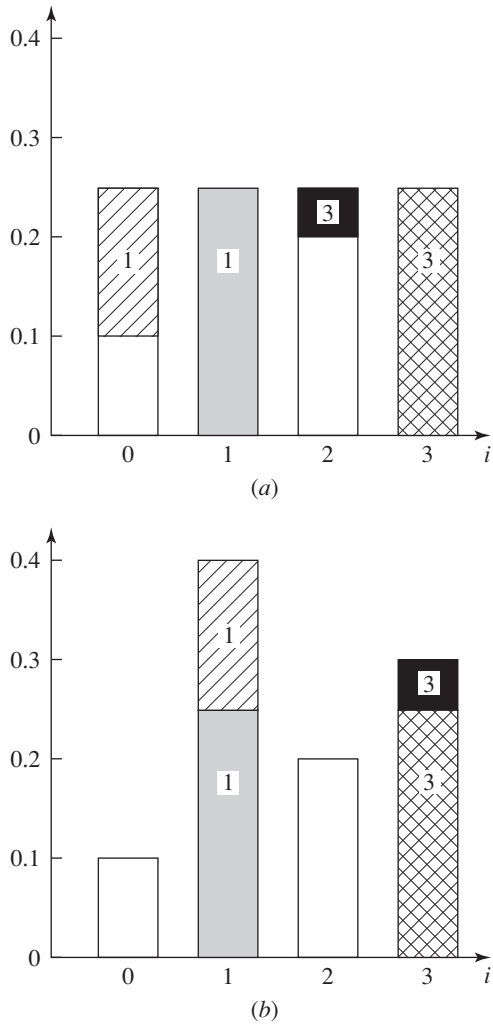


FIGURE 8.14
Setup for the alias method in Example 8.13.

composition method. For example, if X can be any nonnegative integer, we can examine the $p(i)$'s to find an n such that $q = \sum_{i=0}^{n} p(i)$ is close to 1, so that the probability is high that $X \in S_n$. Since for any i we can write

$$p(i) = q\left[\frac{p(i)}{q} I_{S_n}(i)\right] + (1 - q)\left\{\frac{p(i)}{1 - q}[1 - I_{S_n}(i)]\right\}$$

we obtain the following general algorithm:

1. Generate $U \sim \mathrm{U}(0, 1)$. If $U \leq q$, go to step 2. Otherwise, go to step 3.
2. Use the alias method to return X on S_n with probability mass function $p(i)/q$ for $i = 0, 1, \ldots, n$.
3. Use any other method to return X on $\{n + 1, n + 2, \ldots\}$ with probability mass function $p(i)/(1 - q)$ for $i = n + 1, n + 2, \ldots$.

In step 3, we could use the inverse-transform method, for example. Since n was chosen to make q close to l, we would expect to avoid step 3 most of the time.

Finally, we note that all the table-based methods discussed above, as well as the alias method, require some effort in an initial setup stage. Thus, they could be unattractive if the probability mass function changes frequently over time as the simulation proceeds. An efficient method for general discrete-variate generation in this case was developed by Rajasekaran and Ross (1993).

8.4.4 Binomial

To generate a bin(t, p) random variate, recall from Sec. 6.2.3 that the sum of t IID Bernoulli(p) random variables has the bin(t, p) distribution. This relation leads to the following convolution algorithm:

1. Generate $Y_1, Y_2, \ldots, Y_t$ as IID Bernoulli(p) random variates.
2. Return $X = Y_1 + Y_2 + \cdots + Y_t$.

Since the execution time of this algorithm is proportional to t, we might want to look for an alternative if t is large. One possibility would be the direct inverse-transform method with an efficient search. Another alternative is the alias method (see Sec. 8.4.3), since the range of X is finite. Finally, algorithms specific to the binomial distribution that are efficient for large t are discussed by Ahrens and Dieter (1974) and Kachitvichyanukul and Schmeiser (1988).

8.4.5 Geometric

The following algorithm is equivalent to the inverse-transform method if we replace U by $1 - U$ in step 2 (see Prob. 8.14):

1. Generate $U \sim$ U(0, 1).
2. Return $X = \lfloor \ln U / \ln (1 - p) \rfloor$.

The constant $\ln (1 - p)$ should, of course, be computed beforehand. If p is near 0, $\ln (1 - p)$ will also be near zero, so that double-precision arithmetic should be considered to avoid excessive roundoff error in the division in step 2. For p near 1, $\ln (1 - p)$ will be a large negative number, which also could cause numerical difficulties; fortunately, for large p it is more efficient to use an altogether different algorithm based on the relationship between geometric and Bernoulli random variables described in Sec. 6.2.3 (see Prob. 8.14).

8.4.6 Negative Binomial

The relation between the negbin(s, p) and geom(p) distributions in Sec. 6.2.3 leads to the following convolution algorithm:

1. Generate $Y_1, Y_2, \ldots, Y_s$ as IID geom(p) random variates.
2. Return $X = Y_1 + Y_2 + \cdots + Y_s$.

This is simple, but its execution time is proportional to s. For large s, consideration might be given to an alternative method discussed in Fishman (1978), which makes

use of a special relationship between the negative binomial, gamma, and Poisson distributions; its efficiency depends on the ability to generate rapidly from the gamma and Poisson distributions. Other alternatives are discussed in Ahrens and Dieter (1974).

8.4.7 Poisson

Our algorithm for generating Poisson(λ) random variates is based essentially on the relationship between the Poisson(λ) and expo($1/\lambda$) distributions stated in Sec. 6.2.3. The algorithm is as follows:

1. Let $a = e^{-\lambda}$, $b = 1$, and $i = 0$.
2. Generate $U_{i+1} \sim$ U(0, 1) and replace b by bU_{i+1}. If $b < a$, return $X = i$. Otherwise, go to step 3.
3. Replace i by $i + 1$ and go back to step 2.

The algorithm is justified by noting that $X = i$ if and only if

$$\sum_{j=1}^{i} Y_j \le 1 < \sum_{j=1}^{i+1} Y_j$$

where $Y_j = (-1/\lambda) \ln U_j \sim$ expo($1/\lambda$) and the Y_j's are independent. That is, $X = \max\{i: \sum_{j=1}^{i} Y_j \le 1\}$, so that $X \sim$ Poisson(λ) by the first comment in the description of the Poisson distribution in Table 6.4.

Unfortunately, this algorithm becomes slow as λ increases, since a large λ means that $a = e^{-\lambda}$ is smaller, requiring more executions of step 2 to bring the cumulative product of the U_{i+1}'s down under a. [In fact, since X is 1 less than the number of U_{i+1}'s required, the expected number of executions of step 2 is $E(X) + 1 = \lambda + 1$, so that execution time grows with λ in an essentially linear fashion.] One alternative would be to use the alias method in concert with the composition approach (since the range of X is infinite), as described in Sec. 8.4.3. Another possibility would be the inverse-transform method with an efficient search. Atkinson (1979b, 1979c) examined several such search procedures and reported that an indexed search similar to the method of Chen and Asau (1974), discussed earlier in Sec. 8.3.16, performed well. (This search procedure, called PQM by Atkinson, requires a small amount of setup and extra storage but is still quite simple to implement.) Other fast methods of generating Poisson variates are given by Devroye (1981) and by Schmeiser and Kachitvichyanukul (1981).

8.5 GENERATING RANDOM VECTORS, CORRELATED RANDOM VARIATES, AND STOCHASTIC PROCESSES

So far in this chapter we have really considered generation of only a single random variate at a time from various *univariate distributions*. Applying one of these algorithms repeatedly with independent sets of random numbers produces a sequence of IID random variates from the desired distribution.

In some simulation models, however, we may want to generate a random *vector* $\mathbf{X} = (X_1, X_2, \ldots, X_d)^T$ from a specified *joint* (or *multivariate*) *distribution*, where the individual components of the vector might not be independent. (A^T denotes the transpose of a vector or matrix A.) Even if we cannot specify the exact, full joint distribution of $X_1, X_2, \ldots, X_d$, we might want to generate them so that the individual X_i's have specified univariate distributions (called the *marginal distributions* of the X_i's) and so that the correlations, ρ_{ij}, between X_i and X_j are specified by the modeler. In Sec. 6.10 we discussed the need for modeling these situations, and in this section we give examples of methods for generating such correlated random variates and processes in some specific cases. There are several other problems related to generating correlated random variates that we do not discuss explicitly, e.g., generating from a multivariate exponential distribution; we refer the reader to Johnson (1987), Johnson, Wang, and Ramberg (1984), Fishman (1973a, 1978), Mitchell and Paulson (1979), Marshall and Olkin (1967), and Devroye (1997).

8.5.1 Using Conditional Distributions

Suppose that we have a fully specified joint distribution function $F_{X_1, X_2, \ldots, X_d}(x_1, x_2, \ldots, x_d)$ from which we would like to generate a random vector $\mathbf{X} = (X_1, X_2, \ldots, X_d)^T$. Also assume that for $i = 2, 3, \ldots, d$ we can obtain the *conditional distribution* of X_i given that $X_j = x_j$ for $j = 1, 2, \ldots, i - 1$; denote the conditional distribution function by $F_i(x_i | x_1, x_2, \ldots, x_{i-1})$. [See any probability text, such as Mood, Graybill, and Boes (1974, chap. IV) or Ross (2003, chap. 3) for a discussion of conditional distributions.] In addition, let $F_{X_i}(x_i)$ be the marginal distribution function of X_i for $i = 1, 2, \ldots, d$. Then a general algorithm for generating a random vector $\mathbf{X}$ with joint distribution function $F_{X_1, X_2, \ldots, X_d}$ is as follows:

1. Generate X_1 with distribution function F_{X_1}.
2. Generate X_2 with distribution function $F_2(\cdot | X_1)$.
3. Generate X_3 with distribution function $F_3(\cdot | X_1, X_2)$.

.
.
.

d. Generate X_d with distribution function $F_d(\cdot | X_1, X_2, \ldots, X_{d-1})$.
***d* + 1.** Return $\mathbf{X} = (X_1, X_2, \ldots, X_d)^T$.

Note that in steps 2 through d the conditional distributions used are those with the previously generated X_i's; for example, if x_1 is the value generated for X_1 in step 1, the conditional distribution function used in step 2 is $F_2(\cdot | x_1)$, etc. Proof of the validity of this algorithm relies on the definition of conditional distributions and is left to the reader.

As general as this approach may be, its practical utility is probably quite limited. Not only is specification of the entire joint distribution required, but also derivation of all the required marginal and conditional distributions must be

carried out. Such a level of detail is probably rarely obtainable in a complicated simulation.

8.5.2 Multivariate Normal and Multivariate Lognormal

The d-dimensional multivariate normal distribution with mean vector $\boldsymbol{\mu} = (\mu_1, \mu_2, \ldots, \mu_d)^T$ and covariance matrix Σ, where the (i, j)th entry is σ_{ij}, has joint density function given in Sec. 6.10.1. Although the conditional-distribution approach of Sec. 8.5.1 can be applied, a simpler method due to Scheuer and Stoller (1962) is available, which uses a special property of the multivariate normal distribution. Since Σ is symmetric and positive definite, we can factor it uniquely as $\Sigma = CC^T$ (called the *Cholesky decomposition*), where the $d \times d$ matrix C is lower triangular. Algorithms to compute C can be found in Fishman (1973a, p. 217), in Press et al. (2007, sec. 2.9), or among the IMSL routines [Rogue Wave (2013)]. If c_{ij} is the (i, j)th element of C, an algorithm for generating the desired multivariate normal vector $\mathbf{X}$ is as follows:

1. Generate $Z_1, Z_2, \ldots, Z_d$ as IID N(0, 1) random variates.
2. For $i = 1, 2, \ldots, d$, let $X_i = \mu_i + \sum_{j=1}^{i} c_{ij} Z_j$ and return $\mathbf{X} = (X_1, X_2, \ldots, X_d)^T$.

To accomplish the univariate normal generation in step 1, see Sec. 8.3.6. In matrix notation, if we let $\mathbf{Z} = (Z_1, Z_2, \ldots, Z_d)^T$, the algorithm is just $\mathbf{X} = \boldsymbol{\mu} + C\mathbf{Z}$; note the similarity with the transformation $X' = \mu + \sigma X$ for generating $X' \sim \mathrm{N}(\mu, \sigma^2)$ given $X \sim \mathrm{N}(0, 1)$.

The multivariate lognormal random vector from Jones and Miller (1966) and Johnson and Ramberg (1978), as discussed in Sec. 6.10.1, can be represented as $\mathbf{X} = (e^{Y_1}, e^{Y_2}, \ldots, e^{Y_d})^T$, where $\mathbf{Y} = (Y_1, Y_2, \ldots, Y_d)^T \sim \mathrm{N}_d(\boldsymbol{\mu}, \Sigma)$. This relation defines the vector generation algorithm:

1. Generate $\mathbf{Y} = (Y_1, Y_2, \ldots, Y_d)^T \sim \mathrm{N}_d(\boldsymbol{\mu}, \Sigma)$.
2. Return $\mathbf{X} = (e^{Y_1}, e^{Y_2}, \ldots, e^{Y_d})^T$.

Note that $\boldsymbol{\mu}$ and Σ are *not* the mean vector and covariance matrix of the desired multivariate lognormal random vector $\mathbf{X}$, but rather are the mean and covariance matrix of the corresponding multivariate normal random vector $\mathbf{Y}$. Formulas for the expected values and variances of the X_i's, as well as the covariances and correlations between them, were given by Eqs. (6.10) to (6.13) in Sec. 6.10.1, and are functions of and $\boldsymbol{\mu}$ and Σ, the mean vector and covariance matrix of the multivariate normal random vector $\mathbf{Y}$. Thus, if we want to generate an observation from a lognormal random vector $\mathbf{X}$ with *given* mean vector $\boldsymbol{\mu}' = E(\mathbf{X}) = (\mu_1', \mu_2', \ldots, \mu_d')^T$ and *given* covariance matrix Σ' with (i, j)th entry $\sigma_{ij}' = \mathrm{Cov}(X_i, X_j)$, we should solve for $\boldsymbol{\mu}$ and Σ in terms of $\boldsymbol{\mu}'$ and Σ' first, before generating the intermediate multivariate normal random vector $\mathbf{Y}$. The formulas are easily obtained with the ith element of $\boldsymbol{\mu}$ being

$$\mu_i = E(Y_i) = \ln\left(\frac{\mu_i'^2}{\sqrt{\mu_i'^2 + \sigma_i'^2}}\right)$$

and the (i, j)th entry of Σ being

$$\sigma_{ij} = \text{Cov}(Y_i, Y_j) = \ln\left(1 + \frac{\sigma'_{ij}}{|\mu'_i \mu'_j|}\right)$$

8.5.3 Correlated Gamma Random Variates

We now come to a case where we cannot write the entire joint distribution but only specify the marginal distributions (gamma) and the correlations between the component random variables of the $\mathbf{X}$ vector. Indeed, there is not even agreement about what the "multivariate gamma" distribution should be. Unlike the multivariate normal case, specification of the marginal distributions and the correlation matrix does not completely determine the joint distribution here.

The problem, then, is as follows. For a given set of shape parameters α_1, $\alpha_2, \ldots, \alpha_d$, scale parameters $\beta_1, \beta_2, \ldots, \beta_d$, and correlations ρ_{ij} $(i = 1, 2, \ldots, d;$ $j = 1, 2, \ldots, d)$, we want to generate a random vector $\mathbf{X} = (X_1, X_2, \ldots, X_d)^T$ so that $X_i \sim \text{gamma}(\alpha_i, \beta_i)$ and $\text{Cor}(X_i, X_j) = \rho_{ij}$. An immediate difficulty is that not all ρ_{ij} values between -1 and $+1$ are theoretically consistent with a given set of α_i's; that is, the α_i's place a limitation on the possible ρ_{ij}'s [see Schmeiser and Lal (1982)]. The next difficulty is that, even for a set of α_i's and ρ_{ij}'s that *are* theoretically possible, there might not be an algorithm that will do the job. For this reason, we must be content with generating correlated gamma random variates in some restricted cases.

One situation in which there *is* a simple algorithm is the bivariate case, $d = 2$. A further restriction is that $\rho = \rho_{12} \geq 0$, that is, positive correlation, and yet another restriction is that $\rho \leq \min\{\alpha_1, \alpha_2\}/\sqrt{\alpha_1 \alpha_2}$. Nevertheless, this does include many useful situations, especially when α_1 and α_2 are close together. (If $\alpha_1 = \alpha_2$, the upper bound on ρ is removed.) Notice that any two positively correlated exponential random variates are included by setting $\alpha_1 = \alpha_2 = 1$. The algorithm, using a general technique developed by Arnold (1967), relies on a special property of gamma distributions:

1. Generate $Y_1 \sim \text{gamma}(\alpha_1 - \rho\sqrt{\alpha_1 \alpha_2}, 1)$.
2. Generate $Y_2 \sim \text{gamma}(\alpha_2 - \rho\sqrt{\alpha_1 \alpha_2}, 1)$ independent of Y_1.
3. Generate $Y_3 \sim \text{gamma}(\rho\sqrt{\alpha_1 \alpha_2}, 1)$ independent of Y_1 and Y_2.
4. Return $X_1 = \beta_1(Y_1 + Y_3)$ and $X_2 = \beta_2(Y_2 + Y_3)$.

This technique is known as *trivariate reduction*, since the three random variates Y_1, Y_2, and Y_3 are "reduced" to the two final random variates X_1 and X_2. Note that the algorithm does not control the joint distribution of X_1 and X_2; this point is addressed by Schmeiser and Lal (1982).

Correlated gamma random variates can also be generated in some less restrictive cases. Schmeiser and Lal (1982) give algorithms for generating bivariate gamma random vectors with any theoretically possible correlation, either positive or negative. Ronning (1977) treats the general multivariate case $(d \geq 2)$ but again restricts consideration to certain positive correlations. Lewis (1983) gives a method for generating negatively correlated gamma variates that have the same shape and scale parameters.

8.5.4 Generating from Multivariate Families

Multivariate versions of the Johnson translation system, discussed in Sec. 6.10.1, can be generated by inverse-transform methods given by Stanfield et al. (1996). The generated vectors will then match the empirical marginal moments from the sample data, and will have correlations between the coordinates that are close to their sample counterparts as long as the marginal distributions are not heavily skewed.

As mentioned in Sec. 6.10.1, Bézier distributions have been generalized to the bivariate case. Wagner and Wilson (1995) give algorithms for generating samples from the corresponding random vectors based on the conditional-distribution method of Sec. 8.5.1. Extension of Bézier distributions to three or more dimensions is described by Wagner and Wilson (1995) as "feasible but cumbersome." Further results and methods concerning bivariate Bézier distributions can be found in Wagner and Wilson (1996a).

8.5.5 Generating Random Vectors with Arbitrarily Specified Marginal Distributions and Correlations

In Sec. 6.10.2 we noted the need to model some input random variables as a random vector with fairly arbitrary marginal distributions and correlation structure, rather than specifying and controlling their entire joint distribution as a member of some multivariate parametric family like normal, lognormal, Johnson, or Bézier. The individual marginal distributions need not be members of the same parametric family; we may even want to specify them to be of different types—continuous, discrete, or mixed. The only constraint is that the correlation structure between them be internally consistent with the form and parameters of the marginal distributions, as discussed by Whitt (1976), i.e., that the correlation structure specified be *feasible*. The modeling flexibility and marginal-distribution-fitting ease of such a setup has obvious appeal, but the question then becomes how to generate observations on random variables with such arbitrary specified marginals and correlations. In the remainder of this subsection, we mention two methods that address this issue.

Hill and Reilly (1994) describe a technique applicable when the marginal distributions are either all discrete or all continuous, involving composition (or random mixing) of distributions known to exhibit the most extreme feasible correlations to achieve the final correlation structure desired. They present specific examples through dimension $d = 3$ when the marginal distributions are uniform, exponential, and discrete uniform.

Cario et al. (2002) develop a method called *normal to anything* (NORTA). In this approach, we transform a multivariate normal random vector to one having a specified set of marginal distributions (including the case where some marginals are continuous, others are discrete, and still others are mixed continuous-discrete) and any feasible correlation structure. Let F_i denote the ith of the d marginal distribution functions desired for the final generated vector $\mathbf{X} = (X_1, X_2, \ldots, X_d)^T$, and let $\rho_{ij}(\mathbf{X}) = \text{Cor}(X_i, X_j)$ be the desired correlations between the components of $\mathbf{X}$. Initially, a standard multivariate normal random vector $\mathbf{Z} = (Z_1, Z_2, \ldots, Z_d)^T$ is

generated (see Sec. 8.5.2) with $Z_i \sim \text{N}(0, 1)$ and correlations $\rho_{ij}(\mathbf{Z}) = \text{Cor}(Z_i, Z_j)$; specification of the $\rho_{ij}(\mathbf{Z})$'s is discussed below. Letting Φ denote the N(0, 1) distribution function, the ith component of $\mathbf{X}$ is then generated as $X_i = F_i^{-1}[\Phi(Z_i)]$. Since $\Phi(Z_i) \sim \text{U}(0, 1)$ [by a basic result known as the *probability integral transform*, as discussed by Mood, Graybill, and Boes (1974, pp. 202–203), for instance], it is clear that X_i will have the desired marginal distribution F_i by the validity of the inverse-transform method for variate generation. The main task in NORTA generation, then, is to find the correlations $\rho_{ij}(\mathbf{Z})$ between the Z_i's that induce the desired correlations $\rho_{ij}(\mathbf{X})$ between the generated X_i's. Cario et al. (2002) give numerical methods for doing so in general, and indicate that these methods are fairly efficient [see also Chen (2001)]. Also, the N(0, 1) distribution function Φ would need to be evaluated numerically, but there are efficient methods for doing this to high accuracy; see, e.g., chap. 26 of Abramowitz and Stegun (1964). Finally, evaluating the inverse distribution function F_i^{-1} could, depending on its form, require a numerical method or search. Thus, NORTA vector generation does require some internal numerical-method computation, though in most cases the burden should not be too great, particularly in comparison with the work involved in a large, complex dynamic simulation. On the other hand, the benefit of NORTA vector generation is its generality and flexibility within a single framework.

Ghosh and Henderson (2002) show that there are sets of marginal distributions with feasible correlation matrix for $d \geq 3$ that the NORTA method cannot generate. In such cases, they show how to modify the initialization phase of the NORTA method so that it will exactly match the marginals and approximately match the desired correlations [see also Ghosh and Henderson (2003, 2009)].

8.5.6 Generating Stochastic Processes

As mentioned in Sec. 6.10.3, some applications require that we generate observations of the "same" random variable as it is observed through time. For example, we might want to generate a sequence of processing times of parts on a machine, or the sizes of incoming messages in a telecommunications system. If the observations of this random variable are assumed to be IID, we just repeatedly sample independently from the appropriate univariate distribution, as discussed in Secs. 8.1 through 8.4.

But if there is some kind of dependence between successive observations (or between observations spaced more than a single time lag apart), this method will not account for correlation to the possible detriment of model validity; see Livny et al. (1993), for example. In parallel to our discussion in Sec. 6.10.3 of specifying and fitting input stochastic processes, we discuss in this subsection how such processes can be generated for a simulation.

AR and ARMA Processes

Autoregressive (AR) and *autoregressive moving-average* (ARMA) models are generated quite obviously from their very definition, given for the case of an AR(p) process in Eq. (6.14) of Sec. 6.10.3. The process must be initialized in a specific way to obtain stationarity; see Box et al. (2008) for complete details. Thus, to generate

an AR(p) process, the defining recursion in Eq. (6.14) is simply implemented mechanistically, and generation of the normally distributed ε_i's proceeds by any method from Sec. 8.3.6. The autocorrelation structure of the generated process is implied by the weighting parameters in the recursion. The marginal distribution of the X_i's is normal, which limits direct application of these processes as simulation input models.

Gamma Processes

Lewis et al. (1989) discuss generation from these processes, including *exponential autoregressive* (EAR) processes. As mentioned in Sec. 6.10.3, the result is an autocorrelated process with gamma marginals, and an autoregressive-type recursive definition and generation procedure.

ARTA Processes

Autoregressive-to-anything (ARTA) processes, developed by Cario and Nelson (1996, 1998) and described in Sec. 6.10.3, allow the modeler to achieve any desired stationary marginal distribution for the generated process, as well as the desired autocorrelation structure out to any specified lag p. The generation of a specified ARTA process proceeds by generating a stationary AR(p) *base process* $\{Z_i\}$ with N(0, 1) marginals, and then transforming it to the desired $\{X_i\}$ input process via Eq. (6.15) in Sec. 6.10.3. As mentioned there, the autocorrelation of the base $\{Z_i\}$ process is determined so that, after the transformation in Eq. (6.15), the autocorrelation structure of $\{X_i\}$ is as desired. The methodology for this is developed in Cario and Nelson (1996), and software for both specification and generation is discussed in Cario and Nelson (1998).

VARTA Processes

Vector-autoregressive-to-anything (VARTA) processes, which were developed by Biller and Nelson (2003) and described in Sec. 6.10.3, provide a methodology for modeling and generating stationary multivariate stochastic processes $\{\mathbf{X}_1, \mathbf{X}_2, \ldots\}$.

8.6 GENERATING ARRIVAL PROCESSES

In this section, we show how to generate the times of arrival $t_1, t_2, \ldots$ for the arrival processes discussed in Sec. 6.12.

8.6.1 Poisson Processes

The (stationary) Poisson process with rate $\lambda > 0$, discussed in Sec. 6.12.1, has the property that the interarrival times $A_i = t_i - t_{i-1}$ (where $i = 1, 2, \ldots$) are IID exponential random variables with common mean $1/\lambda$. Thus, we can generate the t_i's recursively as follows (assume that t_{i-1} has been determined and we want to generate the next arrival time t_i):

1. Generate $U \sim$ U(0, 1) independent of any previous random variates.
2. Return $t_i = t_{i-1} - (1/\lambda) \ln U$.

The recursion starts by computing t_1 (recall that $t_0 = 0$).

This algorithm can be easily modified to generate any arrival process where the interarrival times are IID random variables, whether or not they are exponential. Step 2 would just add an independently generated interarrival time to t_{i-1} in order to get t_i; the form of step 2 as given above is simply a special case for exponential interarrival times.

8.6.2 Nonstationary Poisson Processes

We now discuss how to generate arrival times that follow a nonstationary Poisson process (see Sec. 6.12.2).

It is tempting to modify the algorithm of Sec. 8.6.1 to generate t_i given t_{i-1} by substituting $\lambda(t_{i-1})$ in step 2 for λ. However, this would be incorrect, as can be seen from Fig. 8.15. (This figure might represent traffic arrival rates at an intersection over a 24-hour day.) If $t_{i-1} = 5$, for example, this erroneous "algorithm" would tend to generate a large interarrival time before t_i, since $\lambda(5)$ is low compared with $\lambda(t)$ for t between 6 and 9. Thus, we would miss this upcoming rise in the arrival rate and would not generate the high traffic density associated with the morning rush; indeed, if t_i turned out to be 11, we would miss the morning rush altogether. Kaminsky and Rumpf (1977) illustrate the danger in using other more sophisticated approximations.

Care must be taken, then, to generate a nonstationary Poisson process in a valid way. A general and simple method proposed by Lewis and Shedler (1979), known as *thinning*, can be used. We present a special case of the thinning algorithm that works when $\lambda^* = \max_t\{\lambda(t)\}$ is finite. Briefly, we generate a stationary Poisson process with constant rate λ^* and arrival times $\{t_i^*\}$ (using, for example, the algorithm of Sec. 8.6.1), then "thin out" the t_i^*'s by throwing away (rejecting) each t_i^* as an arrival with probability $1 - \lambda(t_i^*)/\lambda^*$. Thus, we are more likely to accept t_i^* as an arrival if $\lambda(t_i^*)$ is high, yielding the desired property that arrivals will occur more frequently in intervals for which $\lambda(t)$ is high. An equivalent algorithm, in a more

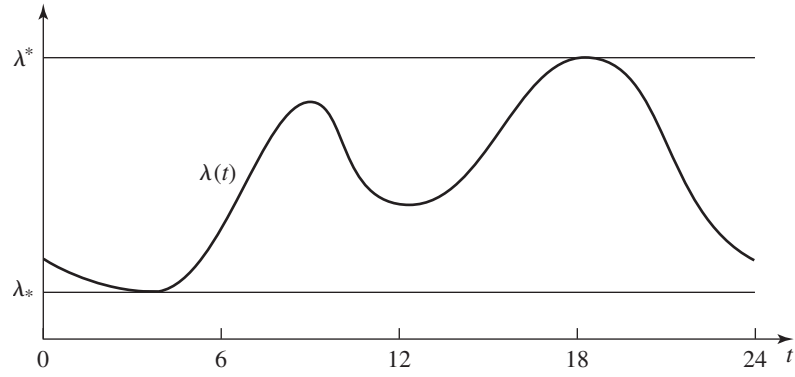


FIGURE 8.15
Nonstationary Poisson process.

convenient recursive form, is as follows (again we assume that t_{i-1} has been validly generated and we want to generate the next arrival time t_i):

1. Set $t = t_{i-1}$.
2. Generate U_1 and U_2 as IID U(0, 1) independent of any previous random variates.
3. Replace t by $t - (1/\lambda^*) \ln U_1$.
4. If $U_2 \le \lambda(t)/\lambda^*$, return $t_i = t$. Otherwise, go back to step 2.

(Once again the algorithm is started by computing t_1.) If the evaluation of $\lambda(t)$ is slow [which might be the case if, for example, $\lambda(t)$ is a complicated function involving exponential and trigonometric calculations], computation time might be saved in step 4 by adding an acceptance pretest; i.e., the current value for t is automatically accepted as the next arrival time if $U_2 \le \lambda_*/\lambda^*$, where $\lambda_* = \min_t \{\lambda(t)\}$. This would be useful especially when $\lambda(t)$ is fairly flat.

EXAMPLE 8.14. Recall Example 6.26, where $\lambda(t)$ was specified empirically from data to be the piecewise-constant function plotted in Fig. 6.57. This rate function is plotted again in Fig. 8.16, along with $\lambda_* = 0.09$ and $\lambda^* = 0.84$ as indicated. Values of $\{t_i^*\}$ from the stationary Poisson process at rate λ^* were generated (using stream 3 of the random-number generator in App. 7A), and are marked by the crosses on the λ^* line; these indeed appear to be distributed uniformly along the line, as would be expected (see Poisson Process Tests in Sec. 6.6). The t_i^*'s were then thinned out, as specified by the algorithm, and the "accepted" arrivals are marked with crosses on the t axis. As desired, these actual arrivals are few and far between when $\lambda(t)$ is low (e.g., between

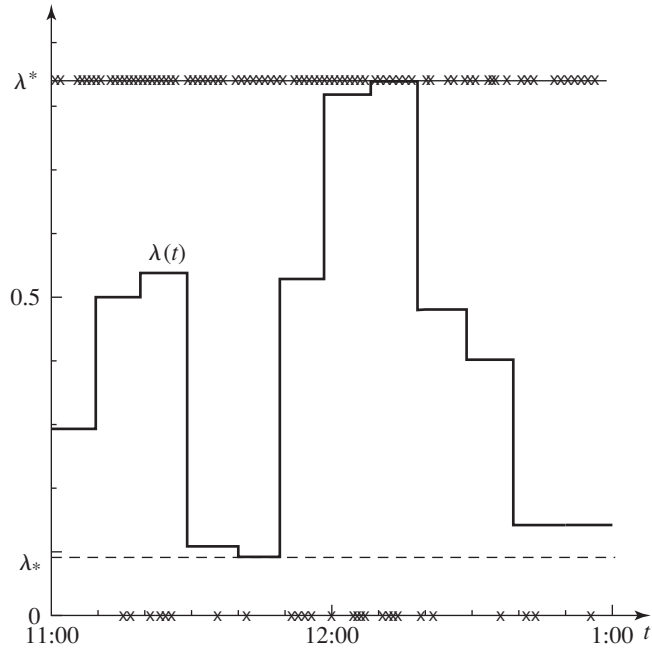


FIGURE 8.16
Generating a nonstationary Poisson process by thinning.

11:30 and 11:50), since most of the t_i^*'s were thinned out. On the other hand, most t_i^*'s were retained as actual arrivals during peak arrival periods (e.g., from 12:00 to 12:20). Note the similarity between Fig. 8.16 and Fig. 8.9, which exemplifies another "thinning" idea, the acceptance-rejection method for random-variate generation.

Although the thinning algorithm is simple, it might be inefficient in some cases. For example, if $\lambda(t)$ is relatively low except for a few high and narrow peaks, λ^* will be a lot larger than $\lambda(t)$ most of the time, resulting in thinning out most of the t_i^*'s. In such cases, a more general thinning algorithm with a nonconstant λ^* curve could be used [see Lewis and Shedler (1979)].

There is a different and older method, treated, for example, by Çinlar (1975, pp. 94–101), and is the analog of the inverse-transform method of random-variate generation, just as the thinning algorithm is analogous to the acceptance-rejection method for variate generation. Recall from Sec. 6.12.2 that the expectation function is

$$\Lambda(t) = \int_0^t \lambda(y)\, dy$$

which will always be a continuous function of t, since it is an indefinite integral; $\Lambda(t)$ is the expected number of arrivals between time 0 and time t. Then a nonstationary Poisson process with expectation function Λ can be generated by first generating Poisson arrival times $\{t_i'\}$ at rate 1, and then setting $t_i = \Lambda^{-1}(t_i')$, where Λ^{-1} is the inverse of the function Λ. Note that *all* of the rate 1 arrival times t_i' are used, in contrast with the thinning method. A recursive version of this algorithm is

1. Generate $U \sim \text{U}(0, 1)$.
2. Set $t_i' = t_{i-1}' - \ln U$.
3. Return $t_i = \Lambda^{-1}(t_i')$.

EXAMPLE 8.15. Figure 8.17*a* plots the expectation function $\Lambda(t)$ corresponding to the rate function $\lambda(t)$ from Fig. 8.16; for comparison purposes, $\lambda(t)$ is redrawn in Fig. 8.17*b*. Note that $\Lambda(t)$ is piecewise linear, since $\lambda(t)$ was specified to be piecewise constant. Also, $\Lambda(t)$ rises most steeply for those values of t where $\lambda(t)$ is highest, i.e., where arrivals should occur rapidly. The times t_i' for the stationary rate 1 Poisson process are plotted on the vertical axis of the plot for $\Lambda(t)$, and do appear to be fairly uniformly spread. Following the dashed lines across to $\Lambda(t)$ and down (i.e., taking Λ^{-1} of the t_i''s) leads to the actual arrival times t_i, marked on the t axis of the plot. The concentration of the t_i's where $\lambda(t)$ is high [and $\Lambda(t)$ is steep, thus "catching" many t_i''s] seems evident, such as between 12:00 and 12:20, and the spreading out of the t_i's during the low-arrival period of 11:30 to 11:50 is also clear. Thus, applying Λ^{-1} to the uniformly spread t_i''s on the vertical axis has the effect of deforming their uniformity to agree with the nonstationarity of the arrival process, just as applying F^{-1} to the uniform U's in the inverse-transform variate-generation algorithm deformed them to agree with the density f. Indeed, there is a strong similarity between Figs. 8.17 and 8.3.

This second algorithm for generating a nonstationary Poisson process does, however, require inversion of Λ, which could be difficult. (In Example 8.15, this could be accomplished easily by a short search and linear interpolation, since Λ was piecewise linear.) This must be traded off against the "wasting" of generated t_i^*'s in the thinning algorithm, which can become particularly inefficient if the rate function

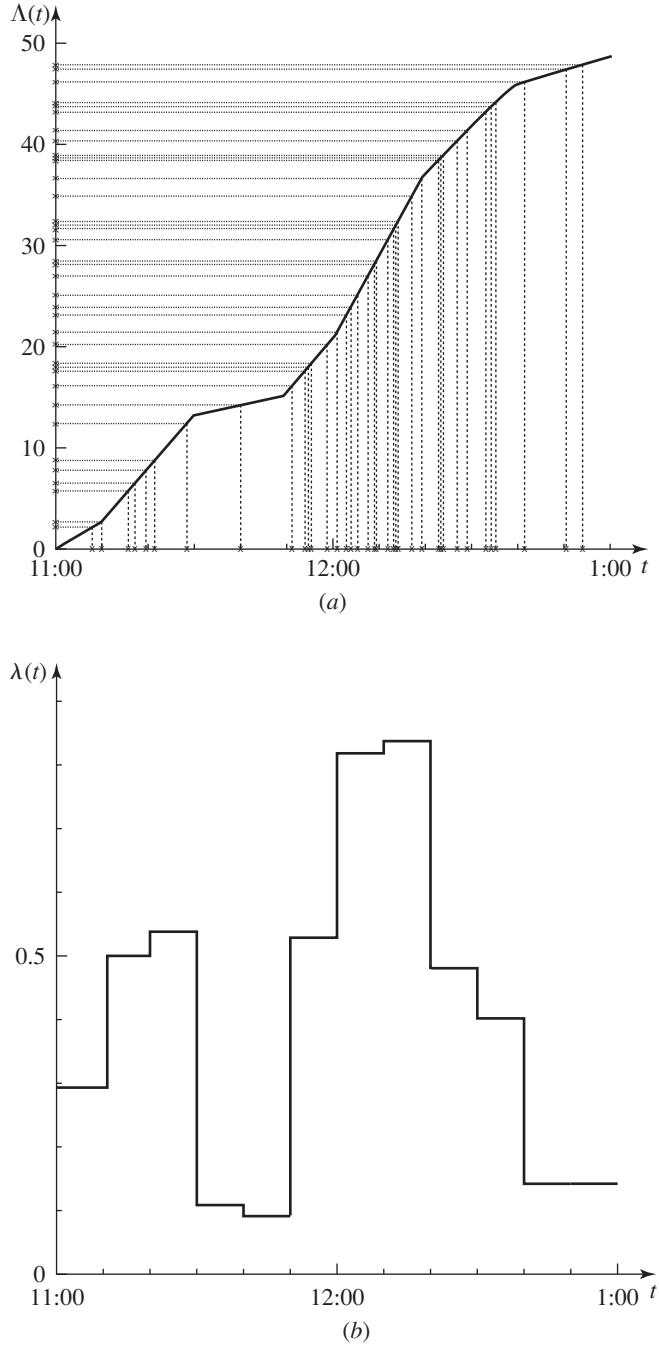


FIGURE 8.17
Generating a nonstationary Poisson process by inverting the expectation function: (*a*) $\Lambda(t)$ and generated arrival times; (*b*) $\lambda(t)$.

$\lambda(t)$ has one or more tall narrow spikes. For more on comparison of these and other methods, see Lewis and Shedler (1979).

Finally, we note that it is sometimes possible to exploit some special feature of the rate function $\lambda(t)$ or cumulative rate function $\Lambda(t)$ to achieve an efficient algorithm. This is the case for some of the specific estimation methods referenced at the end of Sec. 6.12.2, and the interested reader is referred to the original papers for details.

8.6.3 Batch Arrivals

Consider an arrival process where the ith batch of customers arrives at time t_i and the number of customers in this batch is a discrete random variable B_i. Assume that the B_i's are IID and, in addition, are independent of the t_i's. Then a general recursive algorithm for generating this arrival process is as follows:

1. Generate the next arrival time t_i.
2. Generate the discrete random variate B_i independently of any previous B_j's and also independently of $t_1, t_2, \ldots, t_i$.
3. Return the information that B_i customers are arriving at time t_i.

Note that the arrival times $\{t_i\}$ are arbitrary; in particular, they could be from a nonstationary Poisson process.

APPENDIX 8A
VALIDITY OF THE ACCEPTANCE-REJECTION METHOD

We demonstrate here that the acceptance-rejection method for continuous random variables (Sec. 8.2.4) is valid by showing that for any x, $P(X \le x) = \int_{-\infty}^{x} f(y)dy$.

Let A denote the event that acceptance occurs in step 3 of the algorithm. Now X is defined only on the event (or set) A, which is a *subset* of the entire space on which Y and U (of steps 1 and 2) are defined. Thus, *unconditional* probability statements about X alone are really *conditional* probability statements (conditioned on A) about Y and U. Since, given that A occurs we have $X = Y$, we can write

$$P(X \le x) = P(Y \le x | A) \tag{8.2}$$

We shall evaluate the right side of Eq. (8.2) directly.

By the definition of conditional probability,

$$P(Y \le x | A) = \frac{P(A, Y \le x)}{P(A)} \tag{8.3}$$

We shall solve explicitly for the two probabilities on the right side of Eq. (8.3). To do this, it will be convenient first to note that for any y,

$$P(A | Y = y) = P\left[U \le \frac{f(y)}{t(y)}\right] = \frac{f(y)}{t(y)} \tag{8.4}$$

where the first equality follows since U is independent of Y and the second equality since $U \sim \mathrm{U}(0, 1)$ and $f(y) \le t(y)$.

We now use Eq. (8.4) to show that

$$\begin{aligned} P(A, Y \le x) &= \int_{-\infty}^{x} P(A, Y \le x \,|\, Y = y) r(y)\, dy \\ &= \int_{-\infty}^{x} P(A \,|\, Y = y) \frac{t(y)}{c}\, dy \\ &= \frac{1}{c} \int_{-\infty}^{x} f(y)\, dy \end{aligned} \tag{8.5}$$

Next, we note that $P(A) = \int_{-\infty}^{\infty} P(A \,|\, Y = y) r(y)\, dy = 1/c$ [by Eq. (8.4) and the fact that f is a density, so integrates to 1]. This, together with Eqs. (8.5), (8.3), and (8.2), yields the desired result.

APPENDIX 8B
SETUP FOR THE ALIAS METHOD

There are at least two different algorithms for computing the cutoff values F_i and the aliases L_i in the setup for the alias method in Sec. 8.4.3; they do not in general lead to the same sets of cutoff values and aliases for a given distribution, but both will be valid. Originally, Walker (1977) gave the following algorithm in an explicit FORTRAN program:

1. Set $L_i = i$, $F_i = 0$, and $b_i = p(i) - 1/(n + 1)$, for $i = 0, 1, \ldots, n$.
2. For $i = 0, 1, \ldots, n$, do the following steps:
   a. Let $c = \min\{b_0, b_1, \ldots, b_n\}$ and let k be the index of this minimal b_j. (Ties can be broken arbitrarily.)
   b. Let $d = \max\{b_0, b_1, \ldots, b_n\}$ and let m be the index of this maximal b_j. (Ties can be broken arbitrarily.)
   c. If $\sum_{j=0}^{n} |b_j| < \epsilon$, stop the algorithm.
   d. Let $L_k = m$, $F_k = 1 + c(n + 1)$, $b_k = 0$, and $b_m = c + d$.

Note that if the condition in step 2c is satisfied at some point, the rest of the range of i in step 2 will not be completed. This condition in step 2c should theoretically be for equality of the summation to 0, but insisting on this could cause numerical difficulties in floating-point arithmetic; in the above, ϵ is a small positive number such as 10^{-5}.

While the above algorithm is easy to implement in any programming language, Kronmal and Peterson (1979) gave a more efficient algorithm using set operations:

1. Set $F_i = (n + 1)\, p(i)$ for $i = 0, 1, \ldots, n$.
2. Define the sets $G = \{i\colon F_i \ge 1\}$ and $S = \{i\colon F_i < 1\}$.
3. Do the following steps until S becomes empty:
   a. Remove an element k from G and remove an element m from S.
   b. Set $L_m = k$ and replace F_k by $F_k - 1 + F_m$.
   c. If $F_k < 1$, put k into S; otherwise, put k back into G.

This algorithm will leave at least one L_i undefined, but the corresponding F_i values will be equal to 1, so these aliases will never be used in the variate-generation algorithm. Implementing the sets G and S in this second algorithm could be accomplished in many ways, such as a simple push/pop stack or by using a linked-list structure as discussed in Chap. 2.

The second algorithm is more efficient, since in the first algorithm steps 2a and 2b each require a search of $n + 1$ elements, while no such search is required in the second algorithm; this could be important if n is large. However, we should note that numerical difficulties can occur in the second algorithm if the $p(i)$'s do not sum *exactly* to 1; this could occur if, for instance, the $p(i)$'s are proportions corresponding to frequency counts from data, complete with roundoff error. We experienced failure of the second algorithm (using several different set implementations) when the sum of the $p(i)$'s differed from 1 by as little as 10^{-5}.

PROBLEMS

8.1. Give algorithms for generating random variates with the following densities:
(*a*) Cauchy

$$f(x) = \left\{\pi\beta\left[1 + \left(\frac{x-\gamma}{\beta}\right)^2\right]\right\}^{-1} \qquad \text{where } -\infty < \gamma < \infty, \quad \beta > 0, \quad -\infty < x < \infty$$

(*b*) Gumbel (or extreme value)

$$f(x) = \frac{1}{\beta}\exp\left[-e^{-(x-\gamma)/\beta} - \frac{x-\gamma}{\beta}\right] \qquad \text{where } -\infty < \gamma < \infty, \quad \beta > 0, \quad -\infty < x < \infty$$

(*c*) Logistic

$$f(x) = \frac{(1/\beta)e^{-(x-\gamma)/\beta}}{\left[1 + e^{-(x-\gamma)/\beta}\right]^2} \qquad \text{where } -\infty < \gamma < \infty, \quad \beta > 0, \quad -\infty < x < \infty$$

(*d*) Pareto

$$f(x) = \frac{\alpha_2 c^{\alpha_2}}{x^{\alpha_2+1}} \qquad \text{where } c > 0, \quad \alpha_2 > 0, \quad x > c$$

For $\gamma = 0$ and $\beta = 1$ in each of (*a*), (*b*), and (*c*), use your algorithms to generate IID random variates $X_1, X_2, \ldots, X_{5000}$ and write out $\overline{X}(n) = \sum_{i=1}^{n} X_i/n$ for $n = 50, 100, 150, \ldots, 5000$ to verify empirically the strong law of large numbers (Sec. 4.6), i.e., that $\overline{X}(n)$ converges to $E(X_i)$ (if it exists); do the same for (*d*) with $c = 1$ and $\alpha_2 = 2$.

8.2. Let X be discrete with probability mass function $p(1) = 0.05$, $p(2) = 0.05$, $p(3) = 0.1$, $p(4) = 0.1$, $p(5) = 0.6$, and $p(6) = 0.1$, and for $i = 1, 2, \ldots, 6$, let $q(i) = p(1) + p(2) + \cdots + p(i)$. Convince yourself that the following algorithm is explicitly the discrete inverse-transform method with a simple left-to-right search:

1. Generate $U \sim \text{U}(0, 1)$ and set $i = 1$.
2. If $U \le q(i)$, return $X = i$. Otherwise, go to step 3.
3. Replace i by $i + 1$ and go back to step 2.

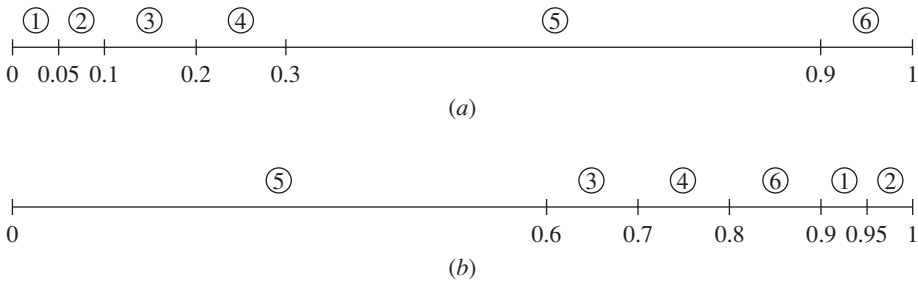


FIGURE 8.18
Representation of the two algorithms in Prob. 8.2.

Let N be the number of times step 2 is executed (so that N is also the number of comparisons). Show that N has the same distribution as X, so $E(N) = E(X) = 4.45$. This algorithm can be represented as in Fig. 8.18*a*, where the circled numbers are the values to which X is set if U falls in the interval directly below them and the search is left-to-right.

Alternatively, we could first sort the $p(i)$'s into decreasing order and form a *coding vector* $i'(i)$, as follows. Let $q'(1) = 0.6$, $q'(2) = 0.7$, $q'(3) = 0.8$, $q'(4) = 0.9$, $q'(5) = 0.95$, and $q'(6) = 1$; also let $i'(1) = 5$, $i'(2) = 3$, $i'(3) = 4$, $i'(4) = 6$, $i'(5) = 1$, and $i'(6) = 2$. Show that the following algorithm is valid:

1′. Generate $U \sim \mathrm{U}(0, 1)$ and set $i = 1$.
2′. If $U \le q'(i)$, return $X = i'(i)$. Otherwise, go to step 3′.
3′. Replace i by $i + 1$ and go back to step 2′.

If N' is the number of comparisons for this second algorithm, show that $E(N') = 2.05$, which is less than half of $E(N)$. This saving in marginal execution time will depend on the particular distribution and must be weighed against the extra setup time and storage for the coding vector $i'(i)$. This second algorithm can be represented as in Fig. 8.18*b*.

8.3. Recall the truncated distribution function F^* and the algorithm for generating from it, as given in Sec. 8.2.1.
(*a*) Show that the algorithm stated in Sec. 8.2.1 is valid when F is continuous and strictly increasing.
(*b*) Show that the following algorithm is also valid for generating X with distribution function F^* (assume again that F is continuous and strictly increasing):

1. Generate $U \sim \mathrm{U}(0, 1)$.
2. If $F(a) \le U \le F(b)$, return $X = F^{-1}(U)$. Otherwise, go back to step 1.

Which algorithm do you think is "better"? In what sense? Under what conditions?

8.4. A truncation of a distribution function F can be defined differently from the F^* of Sec. 8.2.1. Again for $a < b$, define the distribution function

$$\tilde{F}(x) = \begin{cases} 0 & \text{if } x < a \\ F(x) & \text{if } a \le x < b \\ 1 & \text{if } b \le x \end{cases}$$

Find a method for generating from the distribution function $\tilde{F}$, assuming that we already have a method for generating from F. Demonstrate the validity of your algorithm.

8.5. Show that the algorithm in Sec. 8.2.1 for generating the ith order statistic is valid when F is strictly increasing. [*Hint:* Use the fact that if $U_1, U_2, \ldots, U_n$ are IID U(0, 1), then $U_{(i)} \sim \text{beta}(i, n - i + 1)$.] Verify directly that for $i = 1$ and $i = n$ it is valid to let $V = 1 - U^{1/n}$ and $V = U^{1/n}$, respectively.

8.6. Derive the inverse-transform algorithm for the double-exponential distribution of Example 8.3, and compare it with the composition algorithm as given in the example. Which would you prefer?

8.7. For $a < b$, the *right-triangular distribution* has density function

$$f_R(x) = \begin{cases} \dfrac{2(x-a)}{(b-a)^2} & \text{if } a \le x \le b \\ 0 & \text{otherwise} \end{cases}$$

and the *left-triangular distribution* has density function

$$f_L(x) = \begin{cases} \dfrac{2(b-x)}{(b-a)^2} & \text{if } a \le x \le b \\ 0 & \text{otherwise} \end{cases}$$

These distributions are denoted by RT(a, b) and LT(a, b), respectively.

(a) Show that if $X \sim \text{RT}(0, 1)$, then $X' = a + (b - a)X \sim \text{RT}(a, b)$; verify the same relation between LT(0, 1) and LT(a, b). Thus it is sufficient to generate from RT(0, 1) and LT(0, 1).

(b) Show that if $X \sim \text{RT}(0, 1)$, then $1 - X \sim \text{LT}(0, 1)$. Thus it is enough to restrict our attention further to generating from RT(0, 1).

(c) Derive the inverse-transform algorithm for generating from RT(0, 1). Despite the result in (b), also derive the inverse-transform algorithm for generating directly from LT(0, 1).

(d) As an alternative to the inverse-transform method, show that if U_1 and U_2 are IID U(0, 1) random variables, then $\max\{U_1, U_2\} \sim \text{RT}(0, 1)$. Do you think that this is better than the inverse-transform method? In what sense? (See Example 8.4.)

8.8. In each of the following cases, give an algorithm that uses exactly *one* random number for generating a random variate with the same distribution as X.

(a) $X = \min\{U_1, U_2\}$, where U_1 and U_2 are IID U(0, 1).

(b) $X = \max\{U_1, U_2\}$, where U_1 and U_2 are IID U(0, 1).

(c) $X = \min\{Y_1, Y_2\}$, where Y_1 and Y_2 are IID exponential with common mean β.

Compare (a) and (b) with Prob. 8.7. Compare your one-U algorithms in (a) through (c) with the direct ones of actually generating the U_i's or Y_i's and then taking the minimum or maximum.

8.9. The general acceptance-rejection method of Sec. 8.2.4 has the following discrete analog. Let X be discrete with probability mass function $p(x_i)$ for $i = 0, \pm 1, \pm 2, \ldots$, let the majorizing function be $t(x_i) \ge p(x_i)$ for all i, let $c = \sum_{i=-\infty}^{\infty} t(x_i)$, and let $r(x_i) = t(x_i)/c$ for $i = 0, \pm 1, \pm 2, \ldots$.

1′. Generate Y having probability mass function r.

2′. Generate $U \sim \text{U}(0, 1)$, independent of Y.

3′. If $U \le p(Y)/t(Y)$, return $X = Y$. Otherwise, go back to step 1′ and try again.

Show that this algorithm is valid by following steps similar to those in App. 8A. What considerations are important in choosing the function $t(x_i)$?

8.10. For the general acceptance-rejection method (either continuous, as in Sec. 8.2.4, or discrete, as in Prob. 8.9) find the distribution of the number of (Y, U) pairs that are rejected before acceptance occurs. What is the expected number of rejections?

8.11. Give inverse-transform, composition, and acceptance-rejection algorithms for generating from each of the following densities. Discuss which algorithm is preferable for each density. (First plot the densities.)

(a)

$$f(x) = \begin{cases} \dfrac{3x^2}{2} & \text{if } -1 \le x \le 1 \\ 0 & \text{otherwise} \end{cases}$$

(b) For $0 < a < \frac{1}{2}$,

$$f(x) = \begin{cases} 0 & \text{if } x \le 0 \\ \dfrac{x}{a(1-a)} & \text{if } 0 \le x \le a \\ \dfrac{1}{1-a} & \text{if } a \le x \le 1-a \\ \dfrac{1-x}{a(1-a)} & \text{if } 1-a \le x \le 1 \\ 0 & \text{if } 1 \le x \end{cases}$$

8.12. Recall the polar method of Sec. 8.3.6 for generating N(0, 1) random variates. Show that the probability of "acceptance" of W in step 2 is $\pi/4$, and find the distribution of the number of "rejections" of W before "acceptance" finally occurs. What is the expected number of executions of step 1?

8.13. Give a composition algorithm for generating from the triang(0, 1, m) distribution ($0 < m < 1$) of Sec. 8.3.15. Compare it with the inverse-transform algorithm in Sec. 8.3.15. (*Hint:* See Prob. 8.7.)

8.14. (a) Demonstrate the validity of the algorithm given in Sec. 8.4.5 for generating from the geom(p) distribution. (*Hint:* For a real number x and an integer i, $\lfloor x \rfloor = i$ if and only if $i \le x < i + 1$.) Also verify (with $1 - U$ in place of U) that this *is* the inverse-transform algorithm.

(b) Show that the following algorithm is also valid for generating $X \sim$ geom(p):

1. Let $i = 0$.
2. Generate $U \sim$ U(0, 1) independent of any previously generated U(0, 1) random variates.
3. If $U \le p$, return $X = i$. Otherwise, replace i by $i + 1$ and go back to step 2.

Note that if p is large (close to 1), this algorithm is an attractive alternative to the one given in Sec. 8.4.5, since no logarithms are required and early termination is likely.

8.15. Recall the shifted exponential, gamma, Weibull, lognormal, Pearson types V and VI, and log-logistic distributions discussed in Sec. 6.8. Assuming the ability to generate random variates from the original (unshifted) versions of these distributions, give a

general algorithm for generating random variates from the shifted versions. (Assume that the shift parameter γ is specified.)

8.16. Give an explicit algorithm for generating a variate Y from the density $r(x)$ in Example 8.7; $r(x)$ is plotted in Fig. 8.12. (See also Prob. 8.7.)

8.17. The alias method, as stated in Sec. 8.4.3, requires generating at least two U(0, 1) random numbers—one to generate I in step 1 and the other to determine whether I or its alias is returned in step 2. Show that the following version of the alias method, which requires only one random number, is also valid:

1. Generate $U \sim \text{U}(0, 1)$.
2. Let $V = (n + 1)U$, $I = \lfloor V \rfloor$, and $U' = V - I$.
3. If $U' \le F_I$, return $X = I$. Otherwise, return $X = L_I$.

[*Hint:* What is the joint distribution of I and U'? Although this "trick" does reduce the number of random numbers generated, it is probably not a good idea, since it depends on the low-order (least significant) bits of $V - I$ being "random," which may be doubtful for many (pseudo) random-number generators.]

8.18. The setup algorithms in App. 8B for the alias method produce cutoff values F_i that could actually be equal to 1; this will occur for at least one i if the second algorithm is used.
(*a*) Find a way to alter the cutoff and alias values so that every F_i will be strictly less than 1.
(*b*) With the F_i's all being strictly less than 1, find a way to reduce the storage requirements from $2(n + 1)$ to $n + 1$ by combining the L_i and F_i arrays into a single array of length $n + 1$. Restate the alias algorithm from Sec. 8.4.3 so that it works with this one-array method of holding the aliases and cutoff values.

8.19. For the ratio-of-uniforms method, show that the formulas for $u(z)$ and $v(z)$ in Eq. (8.1) are correct.

8.20. Show that the acceptance region S in Example 8.8 is bounded above and below by the curves $v = u$ and $v = u^2$, respectively.

8.21. Develop a majorizing region T that is closer in shape to the acceptance region S for Example 8.8. What is s/t? Give an algorithm for generating points uniformly in T.

8.22. Develop a ratio-of-uniforms algorithm for the standard normal distribution. Draw the acceptance region S. What is s/t?

8.23. The standard Kumaraswamy distribution has the following density function

$$f(x) = \alpha_1\alpha_2 x^{\alpha_1 - 1}\left(1 - x^{\alpha_1}\right)^{\alpha_2 - 1} \qquad \text{for } 0 < x < 1$$

where $\alpha_1 > 0$ and $\alpha_2 > 0$ are shape parameters. Develop an algorithm for generating random variates from this distribution.